

ESCOLA SENAI “A. JACOB LAFER”
CURSO TÉCNICO EM ELETROELETRÔNICA

Lucas Henrique Macedo De Souza
Lucas Sandrin De Souza
Mauricio Fernandes de Mello
Ricardo Santos Gonçalves
Rodrigo Pericinotto da Conceição Moura da Silva

Sistema De Monitoramento De Energia Elétrica

SANTO ANDRÉ

2025

Lucas Henrique Macedo De Souza
Lucas Sandrin De Souza
Mauricio Fernandes de Mello
Ricardo Santos Gonçalves
Rodrigo Pericinotto da Conceição Moura da Silva

Sistema de Monitoramento De Energia Elétrica

Projeto Integrador apresentado à disciplina de Projetos na Escola SENAI “A. Jacob Lafer”, como requisito à obtenção do grau Técnico em Eletroeletrônica.

Orientador: Antonio Benedito

Co orientador: Everton Jonny

SANTO ANDRÉ

2025

Lucas Henrique Macedo De Souza

Lucas Sandrin De Souza

Mauricio Fernandes de Mello

Ricardo Santos Gonçalves

Rodrigo Pericinotto da Conceição Moura da Silva

Sistema de Monitoramento De Energia Elétrica

Projeto Integrador apresentado à disciplina de Projetos na Escola SENAI “A. Jacob Lafer”, como requisito à obtenção do grau Técnico em Eletroeletrônica.

Orientador: Antonio Benedito

Co orientador: Everton Jonny

Professor _____ visto _____

Professor _____ visto _____

Professor _____ visto _____

Professor _____ visto _____

Santo André, ____ de _____ de _____

RESUMO

O nosso trabalho tem como objetivo o desenvolvimento de um sistema de monitoramento de energia elétrica, visando tornar o sistema acessível para aplicações residenciais, industriais e comerciais. A proposta consiste em uma solução que permite o acompanhamento do consumo de energia e o monitoramento das grandezas elétricas em tempo real, com armazenamento de dados em um banco de dados, por meio de uma interface acessível em dispositivos como notebooks ou celulares. O desenvolvimento foi realizado por meio da integração de módulos de medição, comunicação e armazenamento de dados, utilizando a linguagem de programação C++, permitindo o monitoramento de grandezas elétricas como fator de potência, tensão e corrente. O sistema possibilita identificar desperdícios de energia, otimizar o consumo e fornecer informações, contribuindo para a eficiência energética e maior controle do consumo em diferentes ambientes.

PALAVRAS-CHAVE: Monitoramento, Consumo, Energia elétrica e Economizar

SUMÁRIO

1	INTRODUÇÃO	7
1.1	Problema	8
1.2	Justificativa	8
1.3	Objetivos	9
2	INTRODUÇÃO TEÓRICA	10
2.1	Finder 7m38	11
2.2	Microcontrolador ESP32	11
2.3	Módulo MAX3485	12
2.4	Módulo RTC	12
2.5	Módulo MicroSD	12
3	VIABILIDADE TÉCNICA - ECONÔMICA	13
4	PLANEJAMENTO	14
4.1	Solução Inicial	15
4.2	Diagrama em blocos do projeto	16
4.3	Descritivo de funcionamento	17
4.4	Fluxograma da programação	18
4.5	Planejamento – Gráfico de Gantt	19
5	RESULTADOS	21
5.1	Diagrama Eletrônico	21
5.2	Diagramas Elétricos	22
5.3	Layouts das placas / Desenhos mecânicos	24
5.4	Lista De Materiais - Elétrico	28
5.5	Lista De Materiais - Eletrônico	29
5.6	Melhorias	29
6	CONSIDERAÇÕES FINAIS	31

7 REFERÊNCIAS	32
APÊNDICE A – GRÁFICO DE GANTT	35
APÊNDICE B – DIAGRAMA ELETRÔNICO (PROTEUS)	36
APÊNDICE C – LAYOUT DA PLACA (PCB)	37
APÊNDICE D – PLACA PCB	38
APÊNDICE E – PAINEL	39
APÊNDICE F – DIAGRAMA ELÉTRICO - POTÊNCIA	40
APÊNDICE G – DIAGRAMA ELÉTRICO - COMANDO	41
APÊNDICE H – CÓDIGO DA PROGRAMAÇÃO	42
ANEXO A – FINDER 7M38.8.400.0212	54
ANEXO B – ESP32 DEVKIT V1	55
ANEXO C – MAX3485	56
ANEXO D – MÓDULO RTC (REAL TIME CLOCK)	57
ANEXO E – MÓDULO MICROSD	58

1 INTRODUÇÃO

O presente trabalho propõe o desenvolvimento de um sistema de monitoramento de energia elétrica, com foco na redução de custos e no aumento da eficiência energética em diferentes ambientes. Segundo dados da Câmara de Comercialização de Energia Elétrica (CCEE, 2025), o consumo de energia no Brasil apresentou um crescimento de 3,9% em 2024 em relação ao ano anterior, reforçando a necessidade de práticas voltadas ao uso racional e ao acompanhamento mais preciso das grandezas elétricas.

A ausência de monitoramento em tempo real ainda representa uma das principais causas de desperdício, dificuldade de gestão e aumento dos gastos elétricos. Com o avanço da eletrônica e o barateamento de dispositivos inteligentes, tornou-se possível desenvolver sistemas compactos, acessíveis e capazes de registrar e analisar dados de forma eficiente.

Neste contexto, trabalhos recentes destacam o uso de microcontroladores, como o ESP32, na construção de soluções de medição e acompanhamento do consumo elétrico. O sistema desenvolvido neste projeto faz uso do ESP32, do conversor MAX3485, de um módulo MicroSD e do Finder 7M38, proporcionando praticidade e acessibilidade para aplicações em fábricas, condomínios e residências. A proposta contribui para uma gestão mais eficiente e sustentável do uso de energia elétrica.

1.1 Problema

Quando a energia elétrica não é monitorada de forma adequada, resulta em dificuldades no controle de gastos, desperdícios e baixa eficiência. Em diversos ambientes, como residências, comércios e indústrias, a falta de ferramentas e equipamentos para o acompanhamento impede que os responsáveis ou a equipe de manutenção em uma indústria identifique padrões de uso e potenciais pontos de melhoria. Esse cenário destaca a necessidade de soluções que possibilitem o monitoramento eficaz do consumo, guardando os dados obtidos durante o tempo e contribuindo para uma gestão mais eficiente e econômica.

Diante desse cenário, torna-se essencial o desenvolvimento de sistemas de monitoramento de energia elétrica que sejam eficientes, acessíveis e capazes de fornecer dados em tempo real, permitindo uma gestão mais consciente e econômica do consumo.

1.2 Justificativa

A busca por um dispositivo capaz de entregar uma eficiência melhor no consumo de energia tem se tornando cada vez mais relevante diante do aumento contínuo no consumo de energia elétrica. Equipamentos que fornecem o relatório do consumo e monitoramento de grandezas elétricas não são populares no mercado brasileiro. A ausência dessas ferramentas acessíveis para o monitoramento impede a identificação de desperdícios e a otimização do consumo. O desenvolvimento de um sistema de monitoramento baseado no Finder 7M38 e no ESP32 oferece uma solução prática, de baixo custo e com grande potencial de aplicação em diferentes ambientes, como indústrias, residências e comércios.

1.3 Objetivos

O objetivo principal deste projeto é desenvolver um sistema de monitoramento de múltiplas grandezas elétricas, como tensão, corrente e outras variáveis relacionadas ao consumo de energia. A proposta visa otimizar a utilização da energia elétrica, com foco na melhoria de processos, como uma linha de produção de fábrica, possibilitando também a aplicação de manutenção preditiva. O projeto tem como objetivo oferecer uma solução eficiente e prática, adequada para atender diferentes tipos de públicos, incluindo residências, estabelecimentos comerciais e indústrias.

- Objetivo principal:

Desenvolver um sistema de monitoramento de grandezas elétricas: tensão, corrente, frequência, fator de potência, entre outros.

- Proposta do Projeto:

A proposta do nosso projeto é monitorar e otimizar a utilização da energia elétrica, focando na melhoria de processos, como em linhas de produção de fábricas.

- Público-alvo:

Indústrias: monitoramento de grandezas elétricas em ambientes industriais.

Residências; monitoramento do consumo doméstico.

Comércio: gestão de energia em lojas, escritórios entre outros.

- Objetivo do Projeto:

Oferecer uma solução eficiente e prática, adequada para diferentes tipos de públicos e necessidades.

2 INTRODUÇÃO TEÓRICA

A energia elétrica desempenha um papel fundamental no desenvolvimento econômico e social, sendo amplamente utilizada em indústrias, residências e comércios. O Brasil consumiu 3,9% mais energia elétrica em 2024 do que em 2023 e ultrapassou, pela primeira vez, a marca de 70 mil megawatts (MW) médios. Diante desse aumento constante, torna-se essencial adotar práticas que promovam o uso racional e eficiente desse recurso (CCEE, 2025).

Segundo a Tractian (2025), o monitoramento do consumo de energia é indispensável para identificar desperdícios e reduzir custos, especialmente em ambientes industriais. Sistemas de medição vêm sendo amplamente aplicados para fornecer dados em tempo real, permitindo maior controle e uma gestão energética mais sustentável.

Iaronka (2019) destaca o desenvolvimento de dispositivos baseados em microcontroladores, como o ESP32, que possibilitam a criação de sistemas compactos e de baixo custo para o monitoramento de energia elétrica. Essas tecnologias alinham-se ao avanço da Internet das Coisas (IoT), tornando o processo de medição mais acessível e eficiente.

Com base nesses conceitos e em pesquisas anteriores, este trabalho fundamenta-se nos princípios de eficiência energética e IoT para desenvolver um sistema de monitoramento voltado à redução de custos e ao uso sustentável da energia em diferentes ambientes.

2.1 Finder 7m38

O Finder 7M38 apresentado no anexo A é o medidor de energia utilizado no projeto para acompanhar as principais grandezas elétricas, como tensão, corrente e potência. Ele realiza essas medições em tempo real, permitindo um monitoramento constante do sistema. As informações coletadas pelo medidor são enviadas para o ESP32, que faz a leitura dos dados por meio do módulo MAX3485. Assim, o conjunto possibilita acompanhar de forma simples e precisa o comportamento elétrico do circuito.

2.2 Microcontrolador ESP32

O ESP32 apresentado no anexo B é o microcontrolador central do projeto e atua como responsável por todo o processo de coleta e gerenciamento dos dados. Ele recebe as informações das grandezas elétricas enviadas pelo Finder 7M38 por meio da comunicação RS-485, garantindo que os valores de tensão, corrente e potência cheguem de forma correta e contínua. Após a leitura, o ESP32 processa esses dados internamente, filtrando e organizando as informações para facilitar seu uso no restante do sistema.

Além disso, o ESP32 registra uma cópia das medições no módulo MicroSD, permitindo armazenar o histórico para consultas futuras. Paralelamente, o microcontrolador envia os dados em tempo real via Wi-Fi para o Node-RED. Nesse ambiente, as informações são tratadas, exibidas e posteriormente exportadas em formato CSV, possibilitando uma análise mais detalhada no Excel. Dessa forma, o ESP32 integra todas as etapas do processo: coleta, processamento, armazenamento e transmissão dos dados elétricos.

2.3 Módulo MAX3485

O módulo MAX3485 apresentado no anexo C é um transceptor RS-485 utilizado para realizar a interface de comunicação entre o ESP32 e o Finder 7M38. Sua função no projeto é possibilitar que o microcontrolador consiga interpretar corretamente os dados enviados pelo medidor, já que o Finder 7M38 utiliza o protocolo RS-485 para transmitir as informações das grandezas elétricas.

2.4 Módulo RTC

O módulo RTC (Real Time Clock) apresentado no anexo D é o componente responsável por manter o registro preciso de data e hora do sistema, mesmo quando o microcontrolador é desligado ou ocorre alguma falha na alimentação. Ele possui um circuito próprio e uma bateria interna, o que garante a continuidade da contagem do tempo independentemente do funcionamento do ESP32.

A presença do RTC é fundamental para o projeto, pois permite que cada dado coletado pelo ESP32 seja marcado com o horário exato em que a medição foi realizada. Isso garante que todas as informações obtidas pelo Finder 7M38, assim como os dados armazenados no módulo microSD ou enviados ao Node-RED, fiquem organizados de forma cronológica.

2.5 Módulo MicroSD

O módulo MicroSD apresentado no anexo E é utilizado para armazenar externamente os dados coletados pelo sistema. Ele permite que o ESP32 registre informações das grandezas elétricas em arquivos no formato .csv, facilitando a organização e a análise dos resultados em softwares como o Excel. Dessa forma, o módulo garante um histórico acessível e bem estruturado das medições realizadas pelo projeto.

3 VIABILIDADE TÉCNICA - ECONÔMICA

O projeto é viável, pois os componentes necessários, como o ESP32, Max3485, módulo MicroSD são de fácil acesso no mercado e possuem baixo custo. O Microcontrolador ESP32 é amplamente utilizado em IOT (internet das coisas), oferecendo capacidade de processamento adequada e suporte a diversas linguagens de programação, mas o microcontrolador sozinho não consegue coletar os dados do Finder 7M38 que se comunica via RS-485, portanto utilizamos o Max3485 para converter os sinais diferenciais em sinais lógicos TTL que o ESP32 pode processar.

Os dados coletados pelo ESP32 são armazenados no MicroSD, e enviados via Wi-Fi para o Node-RED, que organiza as informações e permite sua visualização em dashboards ou exportação para análise, enquanto o Finder 7M38 viabiliza a medição do consumo de energia elétrica, a combinação desses componentes torna a construção do protótipo completamente viável.

Do ponto de vista econômico, o projeto também é viável, já que alguns materiais necessários têm custo acessível se comparados a componentes semelhantes. O nosso público-alvo inclui comércios, residências e indústrias que buscam um monitoramento eficiente do consumo de energia, mas com um investimento reduzido. Claro que existem sistemas de monitoramento melhores e mais sofisticados, mas eles são voltados a grandes indústrias e têm um preço bastante elevado. Nossa proposta se destaca por oferecer algo mais acessível e com um bom custo-benefício.

4 PLANEJAMENTO

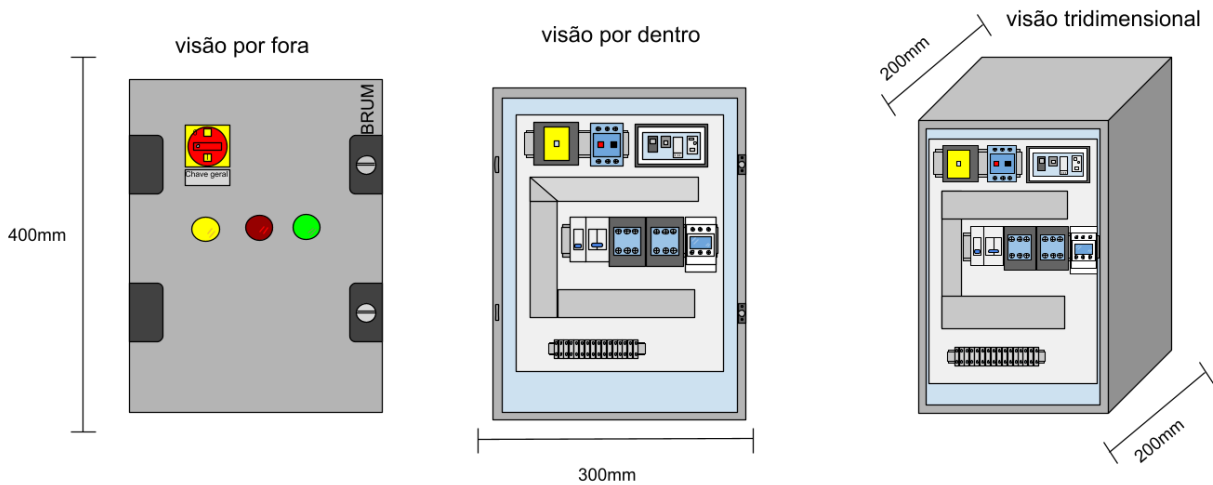
O desenvolvimento do projeto vem sendo realizado em etapas, buscando manter a organização e facilitar a evolução do sistema de monitoramento. No início, foi feita uma pesquisa individual de cada componente, como o ESP32, Finder 7M38, o módulo MicroSD e o MAX3485, para compreender melhor suas funções e verificar a compatibilidade entre eles. Após a etapa de estudo, passamos para os testes dos componentes separadamente. Uma parte do grupo iniciou os testes com o Finder 7M38, a fim de compreender o funcionamento das grandezas exibidas em seu visor e o modo de ligação com uma carga. Em seguida, realizamos testes com o ESP32 e os demais componentes eletrônicos, com o objetivo de avaliar como seria estruturado o protótipo.

Atualmente, estamos avançando na parte mais importante do TCC: a comunicação entre o Finder 7M38, responsável por fornecer as informações, e o ESP32, que realiza o processamento dos dados obtidos. Enquanto uma parte da equipe trabalha na integração entre o medidor e o microcontrolador, a outra está desenvolvendo um painel didático, que será utilizado na apresentação do protótipo e do produto.

4.1 Solução Inicial

Nosso projeto se chama, Sistema de monitoramento de energia elétrica, o coração do nosso projeto será o Finder 7M38, também teremos outros componentes como ESP32, MicroSD e Max 3485. Com esses componentes será possível analisar e monitorar grandezas elétricas como tensão e corrente, podendo evitar possíveis gastos ou perda de produção para indústrias, para demonstrar monitoramento de energia nós estamos montando um painel didático.

Figura 1 – Croqui do protótipo



Símbolo	Descrição
	Finder 7m38.8.400.0212
	Contator tripolar 24VCC
	Disjuntor
	Chave geral
	Disjuntor motor 4-6A
	Borne SAK 2,5mm²
	Sistema de Monitoramento de energia elétrica
	Lâmpada LED amarelo 24VCC

Fonte: Elaborado pelos autores (2025).

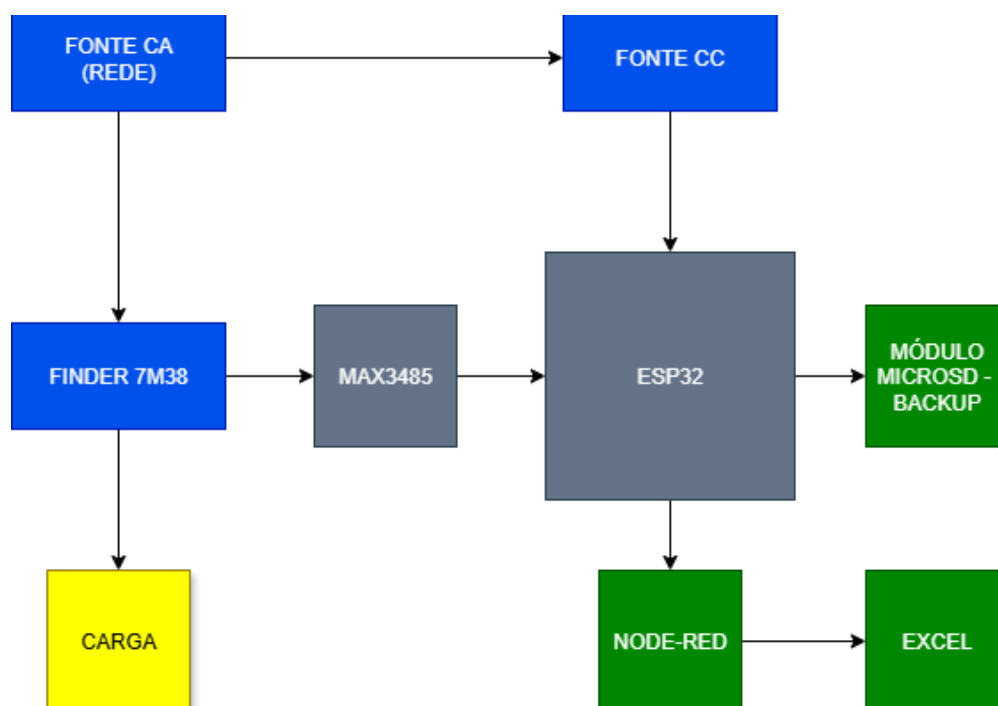
4.2 Diagrama em blocos do projeto

O diagrama abaixo, mostra como será montado o sistema para monitorar consumo de grandezas elétricas. No centro de tudo está o ESP32, um microcontrolador responsável pela coleta e distribuição de dados entre o Módulo MicroSD (Para armazenamento e backup) e o Node-red (visualização em dashboard).

Bom tudo começa com a energia vinda da rede elétrica, que alimenta os principais pontos, o Finder 7M38 que monitora as grandezas elétricas da carga, e

uma fonte CC, que é responsável por alimentar o ESP32, Módulo MicroSD e MAX3485, que desempenha um papel fundamental, pois ele converte sinais de nível de tensão do RS-485 utilizada pelo Finder 7M38, em sinais lógicos TTL que o ESP32 pode processar.

Figura 2 – Diagrama em blocos do projeto.



Fonte: Elaborado pelos autores (2025).

4.3 Descritivo de funcionamento

O primeiro componente a entrar em funcionamento é o Finder 7M38, que é responsável por monitorar as grandezas elétricas, esse dispositivo é conectado entre a rede e carga, realizando o monitoramento de tensão, corrente, potência e tensão de linha.

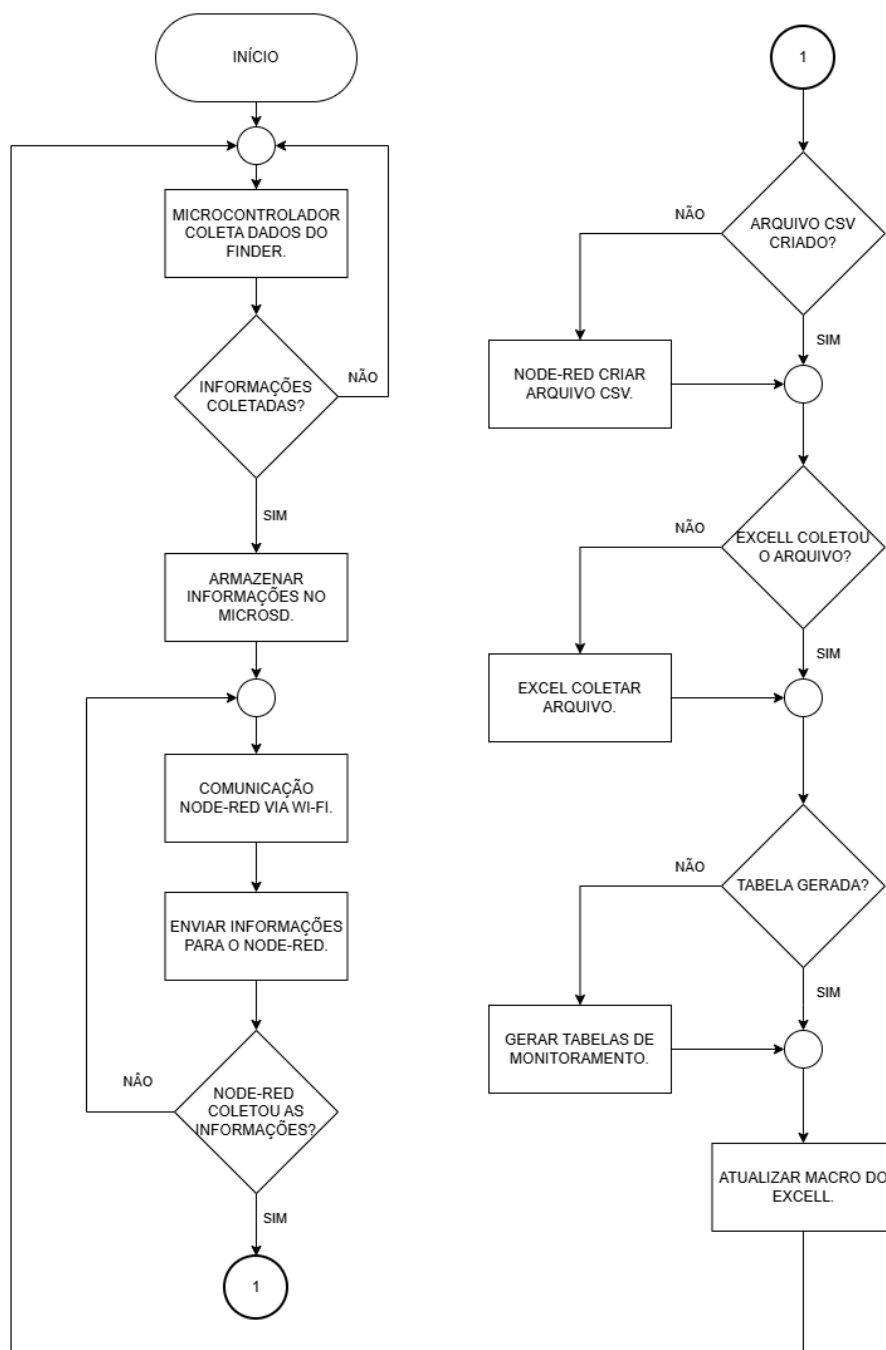
As informações coletadas pelo finder são transmitidas por meio da comunicação RS-485, o ESP32 é incapaz de identificar esse tipo de comunicação transmitido pelo Finder 7M38, para resolver essa limitação vamos utilizar o conversor MAX3485, que converte esses dados diferenciais RS-485 em sinais lógicos TTL, assim o ESP32 consegue processar as informações transmitidas pelo medidor Finder 7M38.

Quando o ESP32 receber esses dados do Finder 7M38, ele vai tratar essas informações coletadas e enviar para o Módulo MicroSD, isso garante que mesmo sem Wi-fi as informações serão armazenadas para uma futura análise dos dados, além de armazenar no MicroSD o ESP32 enviará através do Wi-fi os dados para o Node-RED.

Após o Node-RED receber esses dados, ele criará um arquivo csv, que é um formato de arquivo de texto simples que armazena dados em formato de tabelas, onde cada linha é um registro, com todos os passos anteriores terem sido concluídos, o excel coletará esse arquivo csv, e poderá gerar tabelas de monitoramento e futuramente utilizar um macro para ficar atualizando a tabela em tempo real.

4.4 Fluxograma da programação

Figura 3 – Fluxograma (exemplo).



Fonte – Elaborado pelos autores (2025).

4.5 Planejamento – Gráfico de Gantt

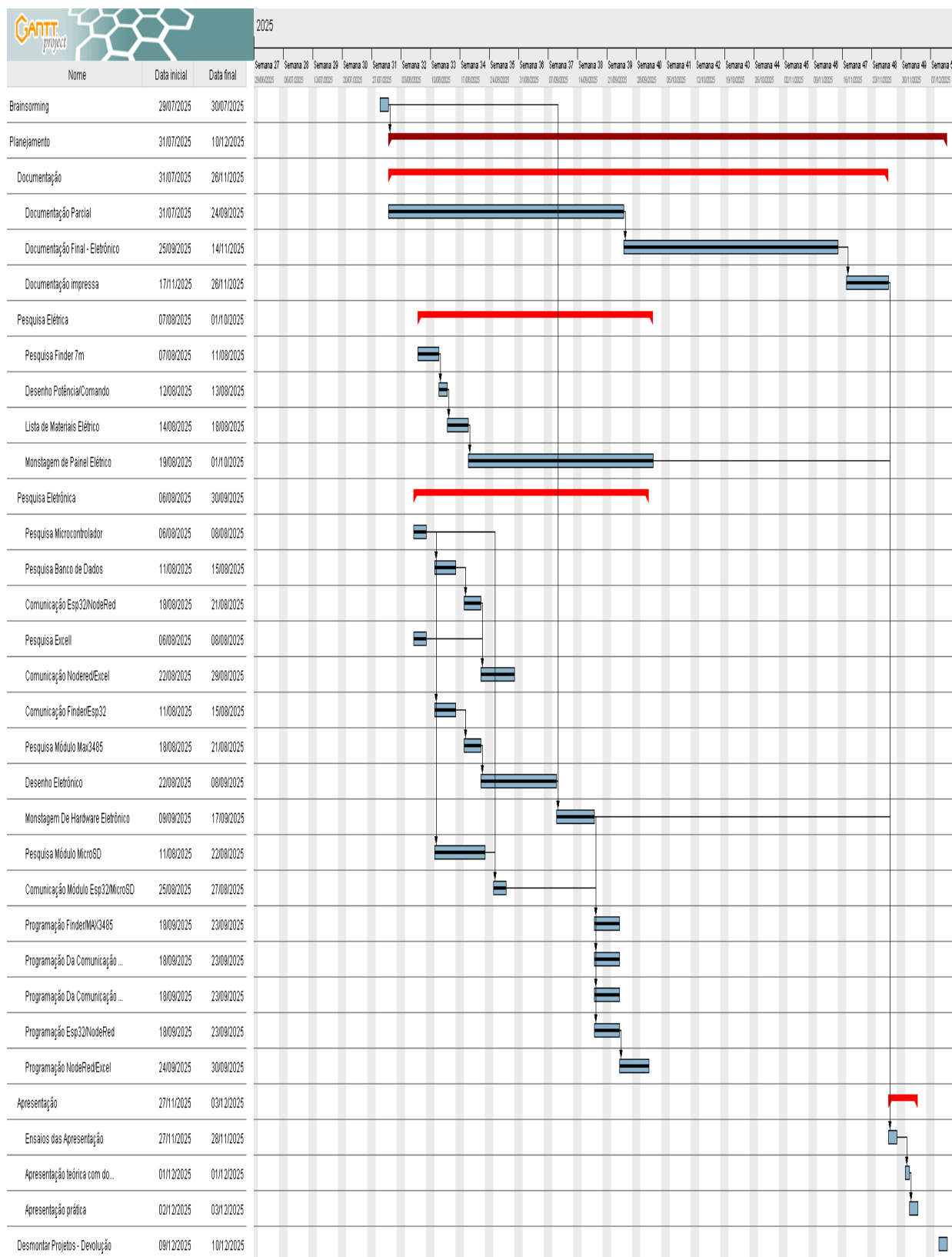
O desenvolvimento do projeto foi organizado em etapas sequenciais e distribuído entre os membros da equipe de forma a otimizar o andamento das atividades. Inicialmente, todos os integrantes realizaram pesquisas técnicas individuais sobre os principais componentes do sistema, incluindo ESP32, Finder 7M38, MAX3485 e Módulo MicroSD. Após essa fase inicial, a equipe foi dividida conforme as demandas específicas do projeto.

Maurício e Lucas Sandrin assumiram as atividades relacionadas à parte elétrica, incluindo testes práticos com o medidor Finder 7M38, elaboração do diagrama elétrico, preparação da lista de materiais e desenvolvimento dos diagramas de potência e comando. Esses integrantes também ficaram responsáveis pela montagem do painel elétrico que será utilizado na apresentação final.

No núcleo eletrônico, todos os membros participaram da implementação inicial dos circuitos e testes. À medida que o trabalho avançou, Lucas Sandrin e Maurício retornaram ao foco principal do painel elétrico, enquanto os demais integrantes continuaram as atividades da eletrônica e da documentação.

Lucas Henrique ficou responsável pela elaboração e organização da documentação, além do monitoramento das etapas do cronograma apresentado no Gráfico de Gantt. Já Rodrigo e Ricardo concentraram-se na programação e no desenvolvimento da comunicação entre o ESP32 e o medidor Finder 7M38.

Quadro 1 – Planejamento – Gráfico de Gantt



Fonte: Elaborado pelos autores (2025)

5 RESULTADOS

Nesta etapa do trabalho, são apresentados os resultados obtidos ao longo do desenvolvimento do sistema de monitoramento de energia. O capítulo demonstra o desempenho do circuito eletrônico montado, a integração entre os componentes (ESP32, MAX3485, Módulo MicroSD e Finder 7M.38) e o funcionamento geral do protótipo em condições simuladas de uso real.

Os testes conduzidos no painel desenvolvido pela equipe permitiram avaliar a estabilidade da comunicação via RS-485, a precisão na leitura do medidor Finder 7M38, o armazenamento dos dados no cartão MicroSD e o envio das informações para o Node-RED através da rede Wi-Fi.

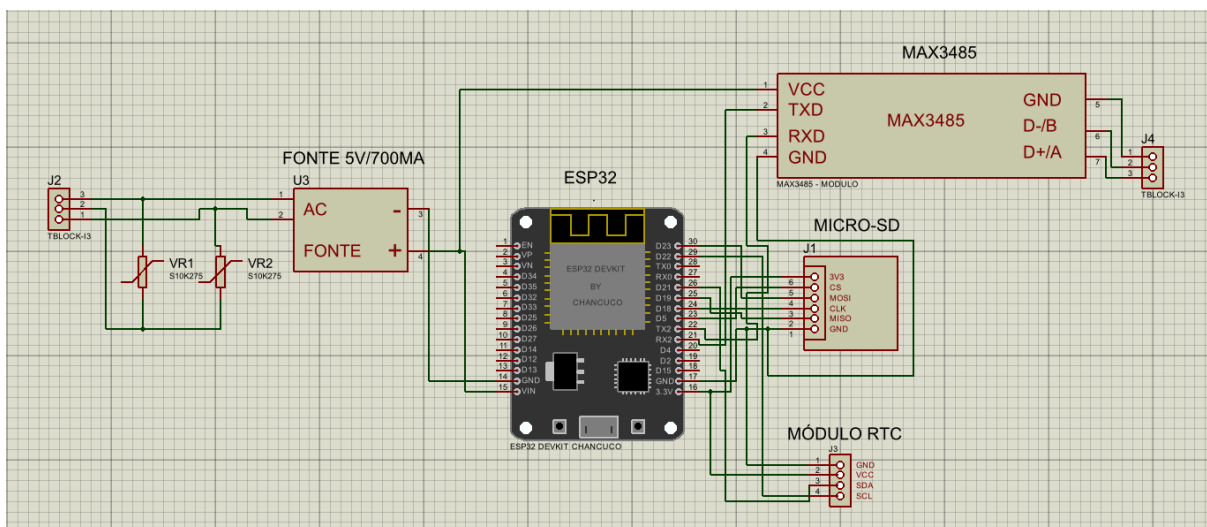
Os resultados mostraram que o sistema opera conforme o esperado, atendendo aos requisitos estabelecidos no início do projeto. A montagem eletrônica corresponde ao planejado, e os testes demonstram que o protótipo é funcional e capaz de representar o comportamento do sistema em ambientes como indústrias, comércios e residências. Este capítulo apresenta, portanto, o desempenho obtido, as validações realizadas e os pontos que confirmam a viabilidade do projeto.

5.1 Diagrama Eletrônico

O diagrama eletrônico apresentado na Figura 4 e no apêndice B representa a montagem completa do sistema de monitoramento de energia elétrica, desenvolvida no software Proteus. O circuito é composto por uma fonte de alimentação de, conectada a uma entrada trifásica e protegida por dois varistores, responsáveis para prevenir curtos-circuitos.

A fonte alimenta o microcontrolador ESP32, que é central de processamento e comunicação. Junto dele, encontra-se o módulo MAX3485, utilizado para a comunicação serial do tipo RS-485, o módulo MicroSD, destinado ao armazenamento dos dados coletados, e o módulo RTC (Real Time Clock), que fornece informações precisas de data e hora para o registro das medições.

Figura 4 - Esquema do projeto completo



Fonte - Elaborado pelos autores (2025).

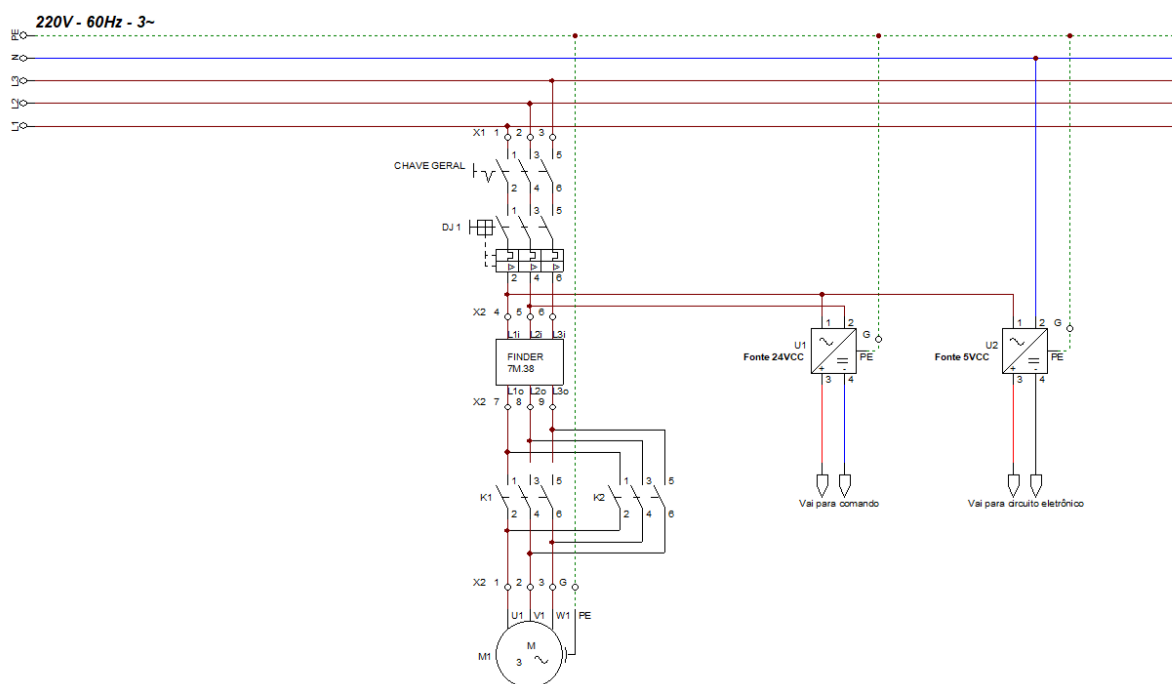
5.2 Diagramas Elétricos

Circuito de potência

O circuito de potência apresentado na figura 5 e no apêndice F é responsável por levar a energia elétrica até o motor trifásico, garantindo o seu funcionamento seguro e eficiente. Ele é composto principalmente pelo Disjuntor motor, que protege contra sobre cargas e curtos-circuitos, e pelo contato tripolar, que realiza o acionamento do motor.

Quando o comando é acionado, o contator fecha seus contatos principais, permitindo a passagem de corrente até o motor. Todo o circuito foi dimensionado para suportar a corrente nominal do motor e assegurar uma operação estável, com proteção adequada e fácil manutenção, além disso, o painel conta com o Finder 7M38, que atua como medidor de energia elétrica, permitindo acompanhar o consumo e o desempenho do sistema durante o funcionamento.

Figura 5 – Circuito de Potência



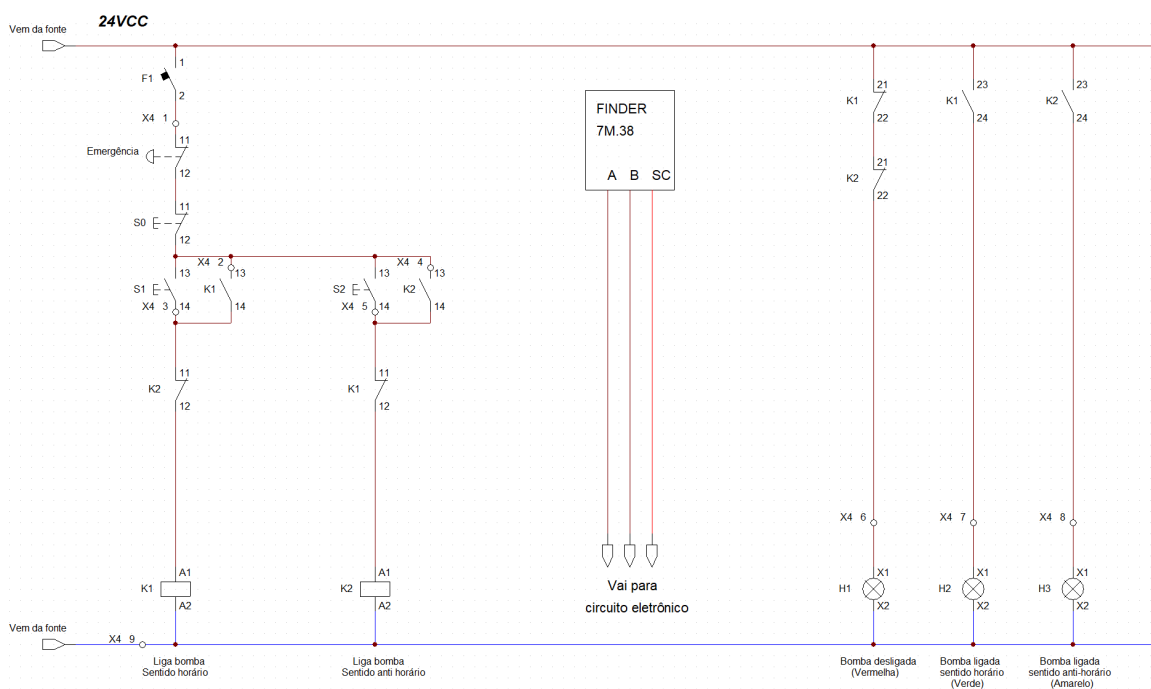
Fonte - Elaborado pelos autores (2025).

Circuito de comando

O circuito de comando apresentado na figura 6 e no apêndice G é responsável por controlar o acionamento e o desligamento do motor em forma prática e segura. Ele é formado pelos botões de liga e desliga, pelo botão de emergência e pelas lâmpadas sinalizadoras, que indicam o estado do sistema durante a operação.

Quando o botão de partida é pressionado, o circuito envia um sinal para o contator da parte de potência, acionando o motor. Já ao pressionar o botão de parada ou em caso de emergência, o circuito interrompe o sinal, desligando o motor.

Figura 6 – Circuito de Comando



Fonte - Elaborado pelos autores (2025).

5.3 Layouts das placas / Desenhos mecânicos

Neste capítulo são apresentados os layouts das placas eletrônicas e os desenhos mecânicos desenvolvidos para o projeto. O objetivo desta seção é mostrar de forma organizada como os componentes foram distribuídos, tanto na parte eletrônica quanto na estrutura física do sistema.

Primeiro, são exibidas as imagens das placas elaboradas, destacando a disposição dos principais módulos, como o ESP32, o MAX3485, o módulo MicroSD e o RTC. O layout mostra a posição dos componentes, as trilhas utilizadas na comunicação entre eles e os pontos de conexão responsáveis pela interface com o medidor Finder 7M38. Essa representação facilita a visualização do funcionamento

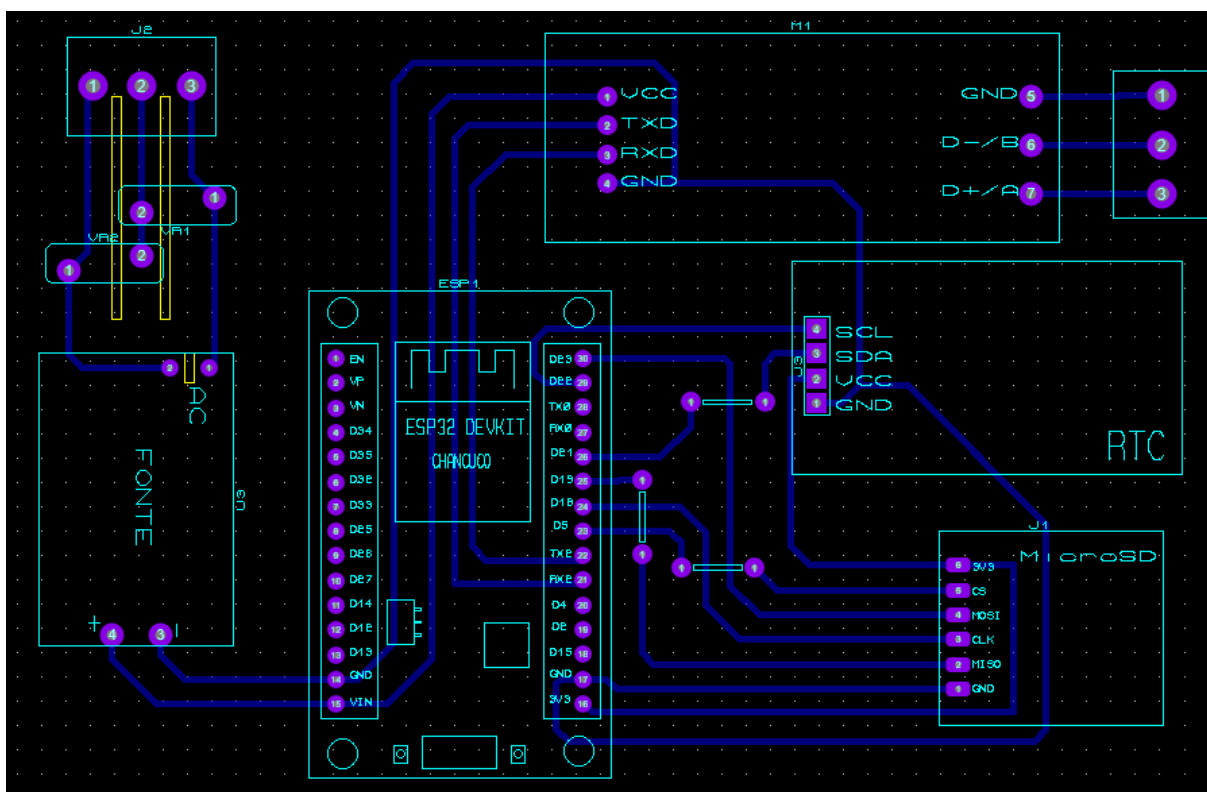
interno do circuito e permite compreender como o sistema foi planejado eletronicamente.

Também são apresentados os desenhos mecânicos relacionados ao painel de testes. Neles é possível observar a estrutura onde os componentes foram fixados, as dimensões adotadas e a organização física necessária para garantir segurança, praticidade e clareza durante os testes de funcionamento. Esses desenhos auxiliam na padronização da montagem e servem como referência para futuras reproduções ou melhorias do protótipo.

A inclusão dos layouts e dos desenhos mecânicos tem como finalidade documentar o projeto de forma completa, permitindo que o leitor compreenda não apenas a lógica do sistema, mas também sua construção física e a relação entre cada etapa da montagem.

A Figura 7 e no apêndice C apresenta o layout da placa (PCB) desenvolvida para o sistema de monitoramento de energia elétrica. O projeto foi elaborado a partir do diagrama eletrônico no software Proteus, sendo posteriormente convertido em trilhas e pads para a confecção da placa.

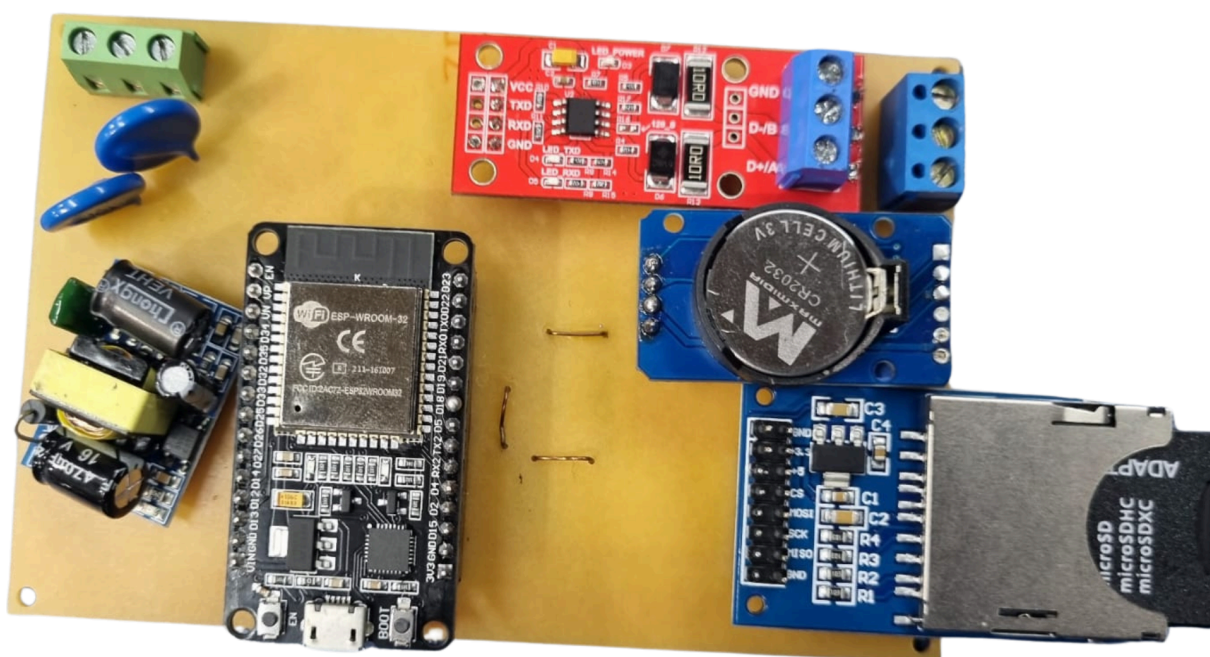
Figura 7 - layout da placa (PCB) desenvolvida no software Proteus



Fonte - Elaborado pelos autores (2025).

A Figura 8 e no apêndice D demonstra o layout da placa (PCB) que usamos no projeto. Primeiro, fizemos todo o desenho no Proteus, montando o esquema e organizando onde cada componente ficaria. Depois disso, pegamos esse layout e fabricamos a placa de verdade, seguindo o que foi planejado no software. O resultado final é a placa física usada no protótipo, com as trilhas e os componentes montados conforme o desenho original.

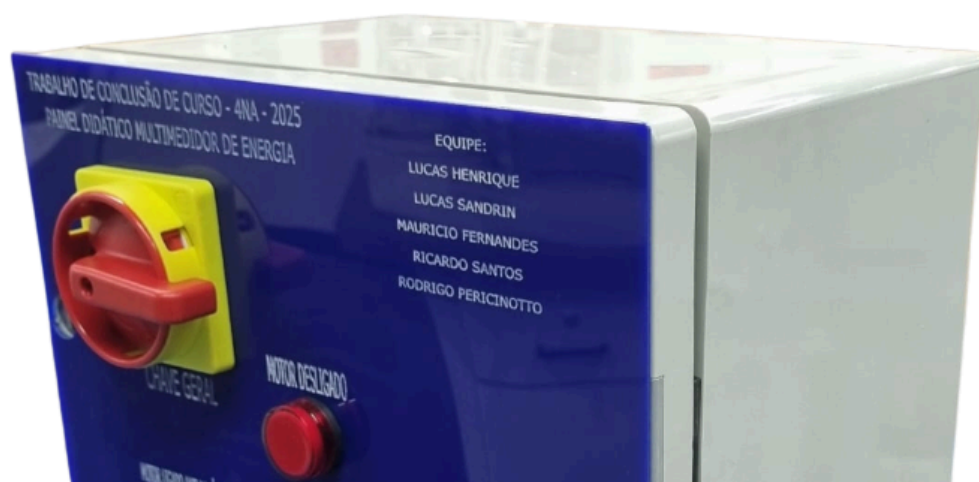
Figura 8 – Placa (PCB)



Fonte - Elaborado pelos autores (2025).

A Figura 9 e no apêndice E apresenta o painel didático utilizado para a demonstração do projeto. Ele foi preparado especialmente para os testes de monitoramento junto com a carga, foram feitas algumas adaptações, substituindo e reorganizando alguns componentes para que o painel atendesse às necessidades do nosso sistema de monitoramento.

Figura 9 – Painel didático



Fonte - Elaborado pelos autores (2025).

5.4 Lista De Materiais - Elétrico

A tabela a seguir, lista os materiais utilizados para a montagem do painel de educacional feito para demonstrar o sistema de monitoramento de energia elétrica. Os componentes foram selecionados com base em critérios de segurança, confiabilidade e compatibilidade elétrica, garantido o correto funcionamento do sistema.

Todos os materiais empregados foram cedidos pelo SENAI, instituição na qual o projeto foi desenvolvido. Dessa forma, não houve custo financeiro para a execução prática do painel.

Item	Qtd	Un	Descrição
1	1	Pç	Chave geral
2	1	Pç	Disjuntor Motor 8 -12 A
3	2	Pç	Contator Tripolar
4	17	Pç	Borne SAK 2,5mm ²
5	1	Pç	Borne Aterramento 2,5mm ²
6	1	Pç	Lâmpada LED Vermelho 24VCC
7	1	Pç	Lâmpada LED Verde 24VCC
8	1	Pç	Lâmpada LED Amarelo 24VCC
9	1	Pç	Botão Pulsador Preto
10	1	Pç	Botão Pulsador Verde
11	1	Pç	Botão Pulsador Vermelho
12	1	Pç	Disjuntor Bipolar 2A
13	1	Pç	Disjuntor Unipolar 4A
14	1	Pç	Botão Emergência
15	5	Metro	Cabo #2,5mm ² Preto
16	2	Metro	Cabo #2,5mm ² Verde
17	15	Metro	Cabo #1,5mm ² Vermelho
18	15	Metro	Cabo #1,5mm ² Branco
19	10	Metro	Cabo #1,5mm ² Azul
20	1	Pç	Placa de Montagem - 350x250mm
21	1	Pç	Painel Elétrico 400x300x200mm
22	1	Pç	Placa de acrílico 400x300
23	50	Cm	Trilho DIN 35x7,5mm
24	8	Pç	Porca Sextavada M5x0,80mm
25	2	Pç	Parafuso M5x100mm
26	6	Pç	Arruela de Pressão 5mm
27	1	Pç	Finder 7m 38.8.400.0212

5.5 Lista De Materiais - Eletrônico

A Tabela a seguir apresenta os componentes eletrônicos utilizados no desenvolvimento do sistema de monitoramento de energia elétrica. Esses materiais foram selecionados com base em sua compatibilidade e necessidade.

Item	Qtd	Un	Descrição	Valor
1	1	Pç	ESP32 Devkit v1	R\$ 30,00 - R\$ 40,00
2	1	Pç	MAX3485	R\$ 26,00 - R\$ 30,00
3	1	Pç	Fonte conversor AC – DC 5V	R\$ 15,00 - R\$ 25,00
4	1	Pç	Módulo MicroSD	R\$ 15,00 - R\$ 20,00
5	1	Pç	Módulo RTC	R\$ 16,00 – R\$ 40,00

5.6 Melhorias

Durante o desenvolvimento do projeto, foram identificadas diversas oportunidades de aprimoramento que podem tornar o sistema mais completo, eficiente e autônomo. Essas melhorias têm como objetivo ampliar as funcionalidades já implementadas e preparar o protótipo para aplicações mais avançadas no futuro.

Uma das principais sugestões é a implementação de um servidor web embarcado diretamente no ESP32. Com isso, as medições poderiam ser acessadas de forma imediata por qualquer navegador, sem depender de plataformas externas como Node-RED. Além disso, seria possível exibir gráficos atualizados em tempo real utilizando WebSocket, bem como permitir a configuração remota de parâmetros do sistema, deixando o monitoramento mais prático e dinâmico.

Outra melhoria importante é a migração do armazenamento em arquivos CSV para um banco de dados dedicado, como InfluxDB ou SQLite. Esse tipo de solução melhora o desempenho nas consultas, facilita análises históricas mais complexas e contribui para uma organização mais eficiente das informações, tornando o sistema mais robusto e escalável.

O projeto também pode ser expandido para integrar funções de controle, indo além do simples monitoramento. A adição de relés de estado sólido, contadores inteligentes ou módulos de comando permitiria executar ações automáticas, como desligamento do motor em situações anormais, redução de carga em horários de

pico ou acionamento remoto do painel. Essa evolução aproximaria o sistema de um supervisor em pequena escala, aumentando significativamente suas possibilidades de aplicação.

Por fim, uma melhoria relacionada à organização dos arquivos no cartão MicroSD consiste na criação automática de registros semanais separados para cada grandeza medida. Essa estrutura reduz o tamanho dos arquivos individuais, facilita a manutenção local e melhora o desempenho das operações de leitura e escrita.

Essas perspectivas de melhorias demonstram que o projeto possui grande potencial de evolução, podendo atender necessidades mais complexas e se adaptar a diferentes cenários de uso no futuro.

6 CONSIDERAÇÕES FINAIS

Considerações Finais Ao comparar o que foi planejado inicialmente com o que foi executado, é possível observar uma evolução significativa no escopo e na maturidade técnica do projeto. No início, o objetivo geral era desenvolver um sistema capaz de monitorar diversas grandezas elétricas fornecidas pelo componente de medição da marca Finder, armazenar esses dados e disponibilizar um dashboard acessível via rede para visualização no computador. Durante a execução, o projeto avançou de forma estruturada e prática, resultando na construção de um painel didático completo, no qual foi integrada a placa baseada em ESP32 responsável pela coleta e transmissão dos dados.

O sistema final inclui: leitura das grandezas elétricas enviadas pelo dispositivo Finder, armazenamento dos dados em cartão SD para garantia de backup, envio das informações para o Node-RED via broker Mosquitto utilizando protocolo MQTT, e exportação automática desses dados para o Excel. Além disso, o ESP32 mantém sincronização de data e hora via rede, garantindo registros precisos. Durante o desenvolvimento, foi necessário adquirir novos conhecimentos técnicos em Modbus, programação do ESP32, implementação de MQTT, estruturação de fluxos no Node-RED e montagem de PCB personalizada. Essas etapas trouxeram desafios, especialmente na comunicação Modbus, na confiabilidade da transmissão MQTT e na organização do armazenamento local em SD, mas também proporcionaram uma evolução técnica consistente. Inicialmente, a intenção era trabalhar com InfluxDB e Grafana para visualização, porém optou-se por um caminho mais adequado ao contexto do projeto: Node-RED integrado ao Excel, garantindo simplicidade, clareza e acessibilidade para uso didático. Como um todo, o projeto alcançou resultados sólidos, apresentando não apenas funcionamento técnico, mas também uma evolução significativa em organização, documentação e integração de tecnologias.

7 REFERÊNCIAS

AUTOCOREROBOTICA. **Módulo RTC (real time clock)**. Disponível em: https://www.autocorerobotica.com.br/real-time-clock-rtc-ds3231?srsId=AfmBOooVqSQXoVSagjuBEUP4pQe1bG6d5e3nAEpUCkD86m6e0ByQb_9N. Acesso em: 24 set. 2025.

CCEE. **Estudo CCEE mostra crescimento de 3,9% no consumo de energia elétrica no Brasil em 2024**. 2025. Disponível em: <https://www.ccee.org.br/pt/web/guest/-/estudo-da-ccee-mostra-crescimento-de-3-9-no-consumo-de-energia-eletrica-no-brasil-em-2024>. Acesso em: 27 fev. 25.

CONRAD. **Finder 7m38**. 2025. Disponível em: <https://www.conrad.com/en/p/finder-7m-38-8-400-0212-7m3884000212-2-way-rail-mount-electricity-meter-digital-80-a-1-pcs2474056.html?srsId=AfmBOoo2WzIfaznpTtAmYwTgOndcpWvolEgW6LNjaWrVv6aQ9OKEAsaB>. Acesso em: 20 set. 2025.

CORDEIRO, Erik. **Porque o monitoramento de energia elétrica é importante**. 2025. Disponível em: <https://traction.com/blog/por-que-o-monitoramento-de-energia-eletrica-e-importante>. Acesso em: 26 abr. 2025.

CÓRDOVA, Gabriela. **Sistema de Monitoramento em Tempo Real do Consumo de Energia**. 2021. Disponível em: <https://repositorio.utfpr.edu.br/jspui/bitstream/1/26704/1/monitoramentoconsumoeenergiaeletrica.pdf>. Acesso em: 7 maio 2021.

SOARES, Fernando. Como levar Internet às áreas remotas do Brasil. **GHZ Ciência e Tecnologia**. Porto Alegre, 4 set. 2020. Disponível em: <https://gauchazh.clicrbs.com.br/tecnologia/noticia/2020/09/como-levar-internet-as-areas-remotas-do-brasil-ckeogzcqb000e014yro2qgssw.html>. Acesso em: 16 mar. 2022. Acesso em: 8 set. 2020.

ELECTRONICPRO. **MAX3485**. Disponível em:
<https://www.electronicpro.co.za/products/5pcs-max3485-ttl-to-rs485-module-mcu-development-converter-module-board-accessories>. Acesso em: 20 set. 2025.

Epe GOV. **Caderno sobre Ações de Eficiência Energética em Indústrias Brasileiras**. 2019. Disponível em:
<https://www.epe.gov.br/pt/publicacoes-dados-abertos/publicacoes/caderno-sobre-acoes-de-eficiencia-energetica-em-industrias-brasileiras>. Acesso em: 13 ago. 2019.

Epe.GOV. **Balanco Energético Nacional 2020**. 2020. Disponível em:
<https://www.epe.gov.br/pt/publicacoes-dados-abertos/publicacoes/balanco-energetico-nacional-2020>. Acesso em: 16 abr. 2020.

ESPBOARDS. **ESP32 Devkitv1**. Disponível em:
<https://www.espboards.dev/esp32/esp32doit-devkit-v1/>. Acesso em: 20 set. 2025.

FINDER. **Protocolo de Comunicação Modbus**. 2025. Disponível em:
https://cdn.findernet.com/app/uploads/2021/09/20090052/Modbus-7M24-7M38_v2_30062021.pdf?utm_source=chatgpt.com. Acesso em: 2 set. 2025.

GITHUB. **Comunicação ESP32 e MAX485**: ESP32 não detecta nenhum dado. 2023. Maialbe2823. Disponível em:
<https://github.com/espressif/esp-idf/issues/10781>. Acesso em: 12 ago. 2025.

GITHUB. **Proteus Library Esp32 Devkit. 2024**. Chancuco. Disponível em:
<https://github.com/CHANCUCO/PROTEUS-LIBRARY-ESP32-DEVKIT>. Acesso em: 27 ago. 2025.

I/O, Esp32. **Comunicação RS485 Com ESP32**. 2025. Disponível em:
<https://esp32io.com/tutorials/esp32-rs485>. Acesso em: 14 set. 2025.

MEDIUM. **Inserir dados do ESP32 no MySQL.** 2023. Tara Chandani Haryono. Disponível em:

<https://medium.com/@tara.haryono/insert-esp32-data-into-mysql-78bfc9893b7>.

Acesso em: 19 ago. 2025.

ROBOCORE. **Real Time Clock DS3231.** 2025. Disponível em:

https://www.robocore.net/tutoriais/primeiros-passos-rtc-ds3231-com-arduino?srsId=AfmBOopj5PMuQnm1y6ONL0VuVUQ8G6zEN3NpLySQaGoQHmj0D5_KpMd.

Acesso em: 24 set. 2025.

SARAVATI. **Módulo MicroSD.** Disponível em:

https://www.saravati.com.br/modulo-leitor-de-cartao-sd-card.html?srsId=AfmBOooR3ODLadiFqqni0FS5Qsl19YKzFOJqf2lIXVm_xPEcJQSDjK0C. Acesso em: 24 set. 2025.

SILVA, Saymon Rafael da Cunha. **SISTEMA DE MONITORAMENTO E CONTROLE DE ENERGIA UTILIZANDO O MICROCONTROLADOR ESP32.** 2021. Disponível em

[:https://repositorio.utfpr.edu.br/jspui/bitstream/1/29084/4/monitoramentocontroleenergiamicrocontrolador.pdf](https://repositorio.utfpr.edu.br/jspui/bitstream/1/29084/4/monitoramentocontroleenergiamicrocontrolador.pdf). Acesso em: 03 dez. 2021.

TURELLA, Rafael. **Monitoramento de Energia: manual completo.** 2025.

Disponível em: <https://www.cubienergia.com/monitoramento-de-energia/>. Acesso em: 28 jan. 2025.

TUTORIAL, Random Nerd. **Guia Módulo de Cartão MicroSD.** 2025. Disponível em:

<https://randomnerdtutorials.com/esp32-microsd-card-arduino/>. Acesso em: 19 ago. 2025.

TUTORIAL, Random Nerd. **Guia Módulo de Cartão MicroSD.** 2025. Disponível em:

<https://randomnerdtutorials.com/esp32-microsd-card-arduino/>. Acesso em: 19 ago. 2025.

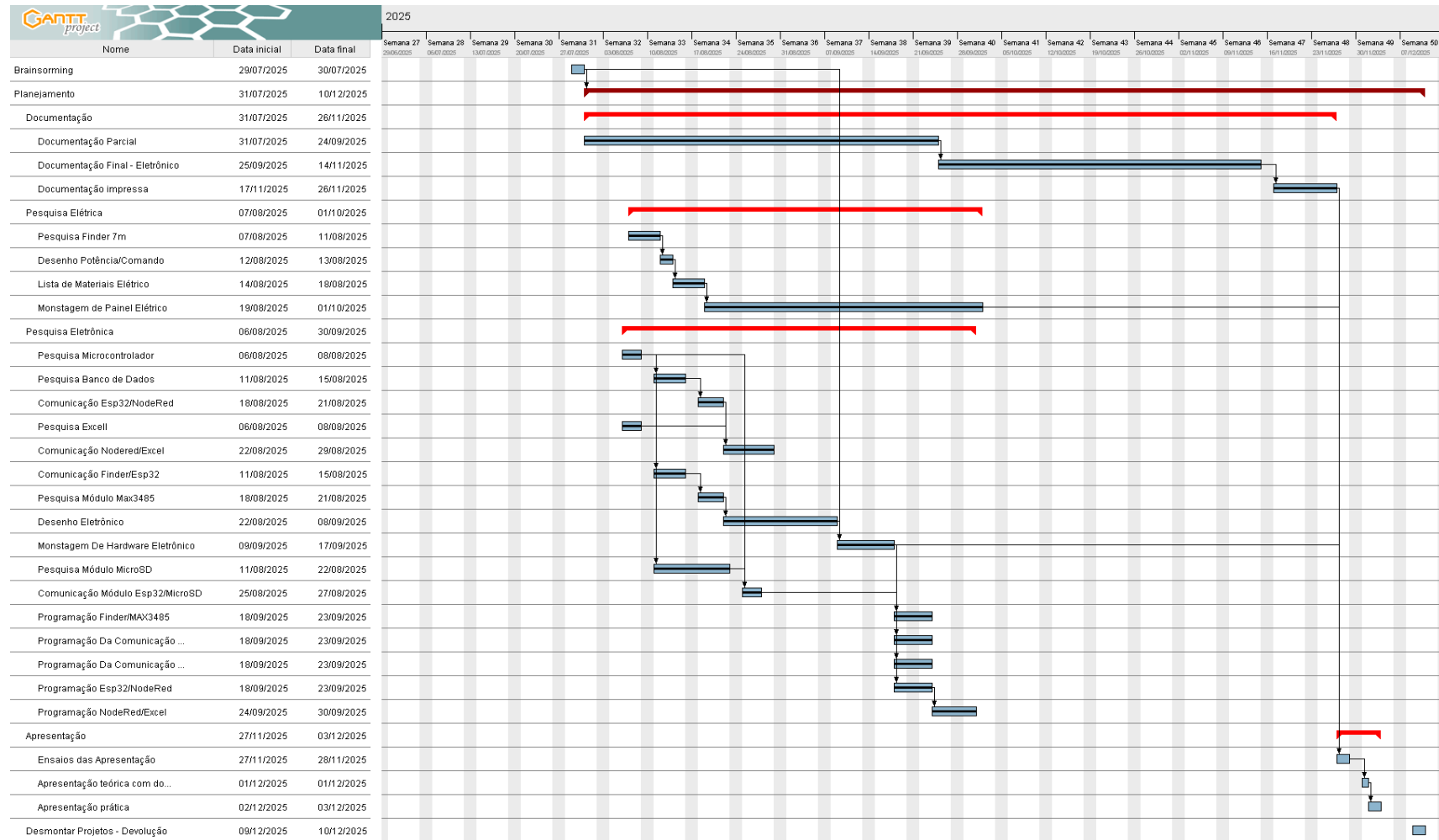
TUTORIAL, Random Nerd. **Guia Módulo de Relógio em Tempo Real.** 2025.

Disponível em:

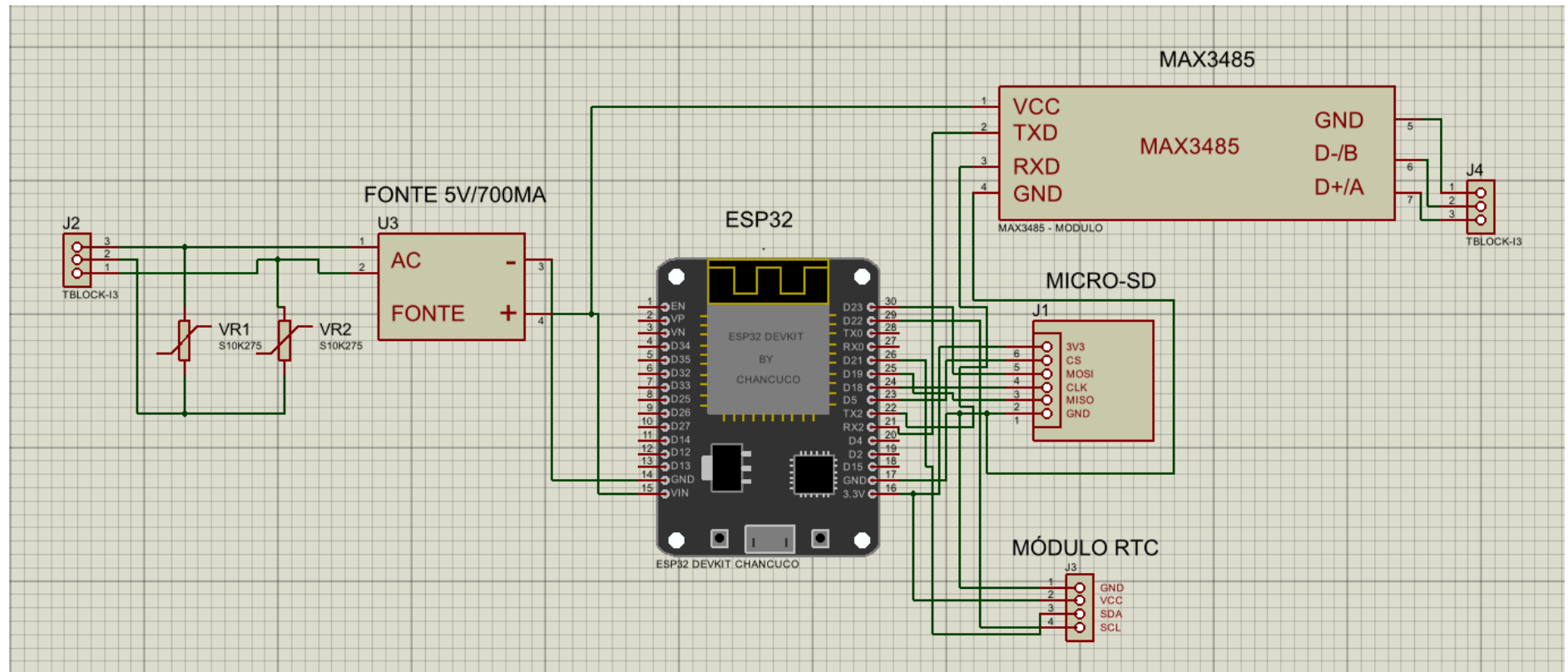
<https://randomnerdtutorials.com/esp32-ds3231-real-time-clock-arduino/>. Acesso em: 8 ago. 2025.

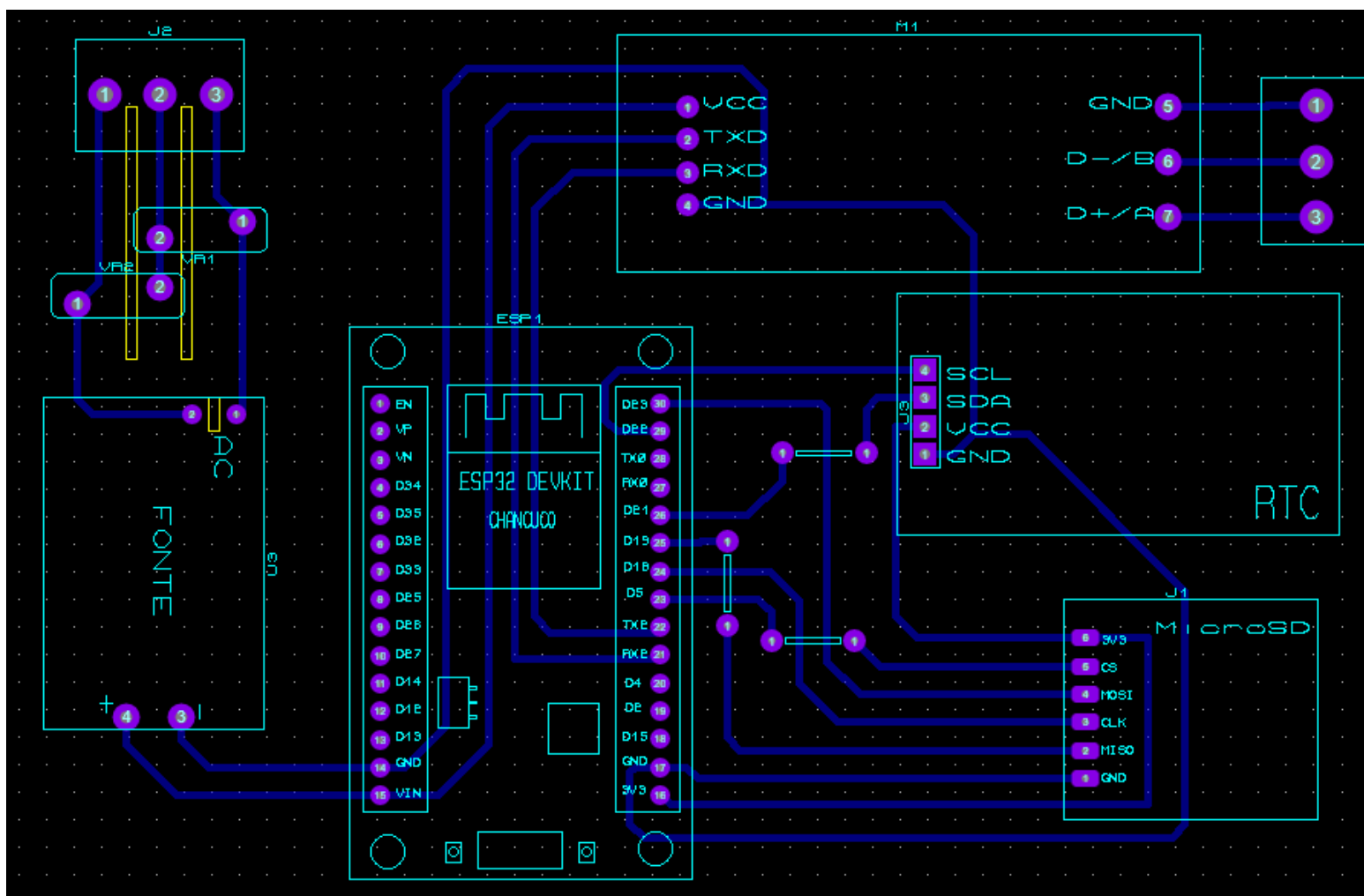
TUTORIALS, Random Nerd. **ESP32: Introdução ao InfluxDB**. 2025. Disponível em: <https://randomnerdtutorials.com/esp32-influxdb/>. Acesso em: 19 ago. 2025.

APÊNDICE A – GRÁFICO DE GANTT

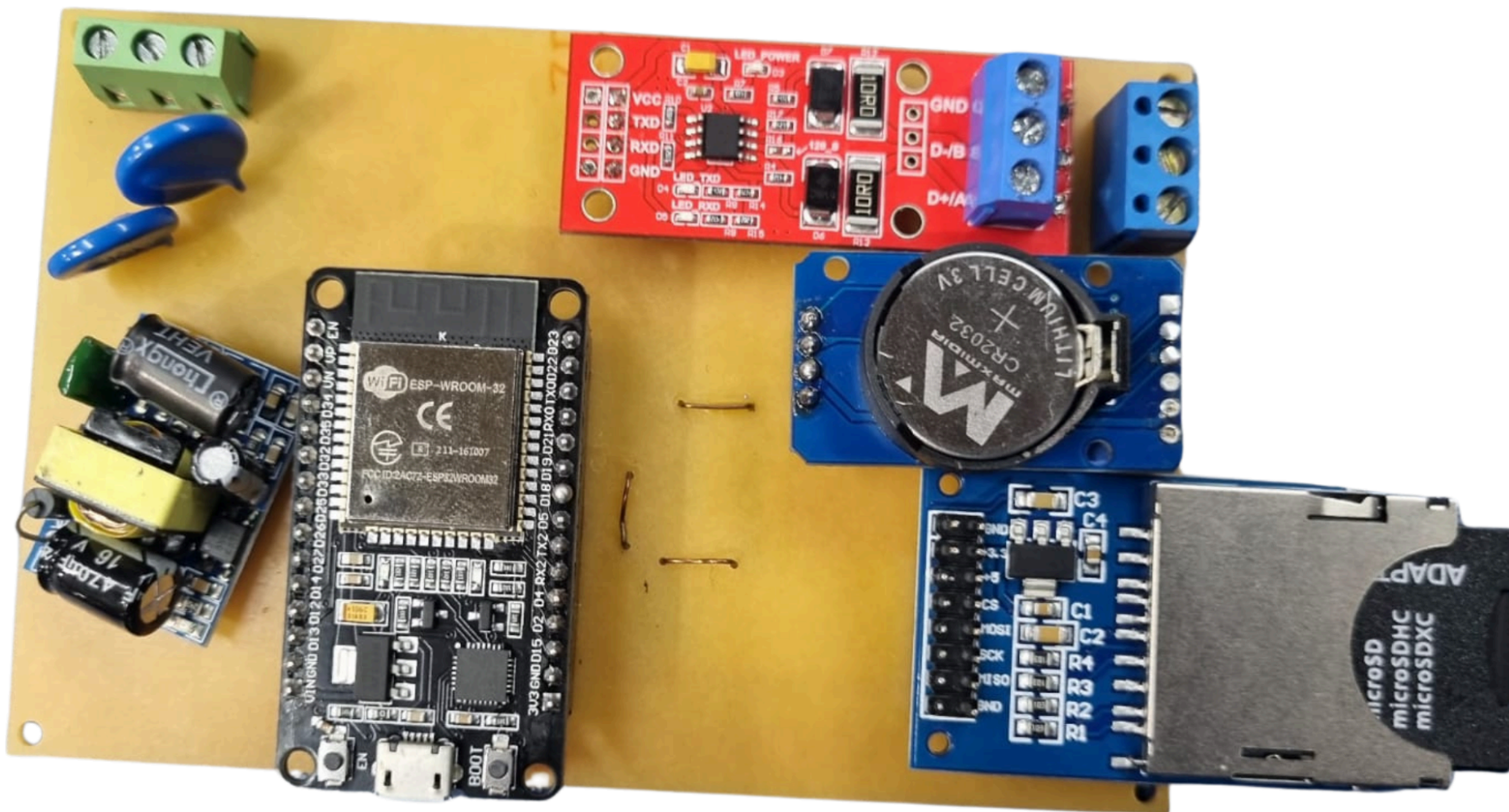


APÊNDICE B – DIAGRAMA ELETRÔNICO (PROTEUS)





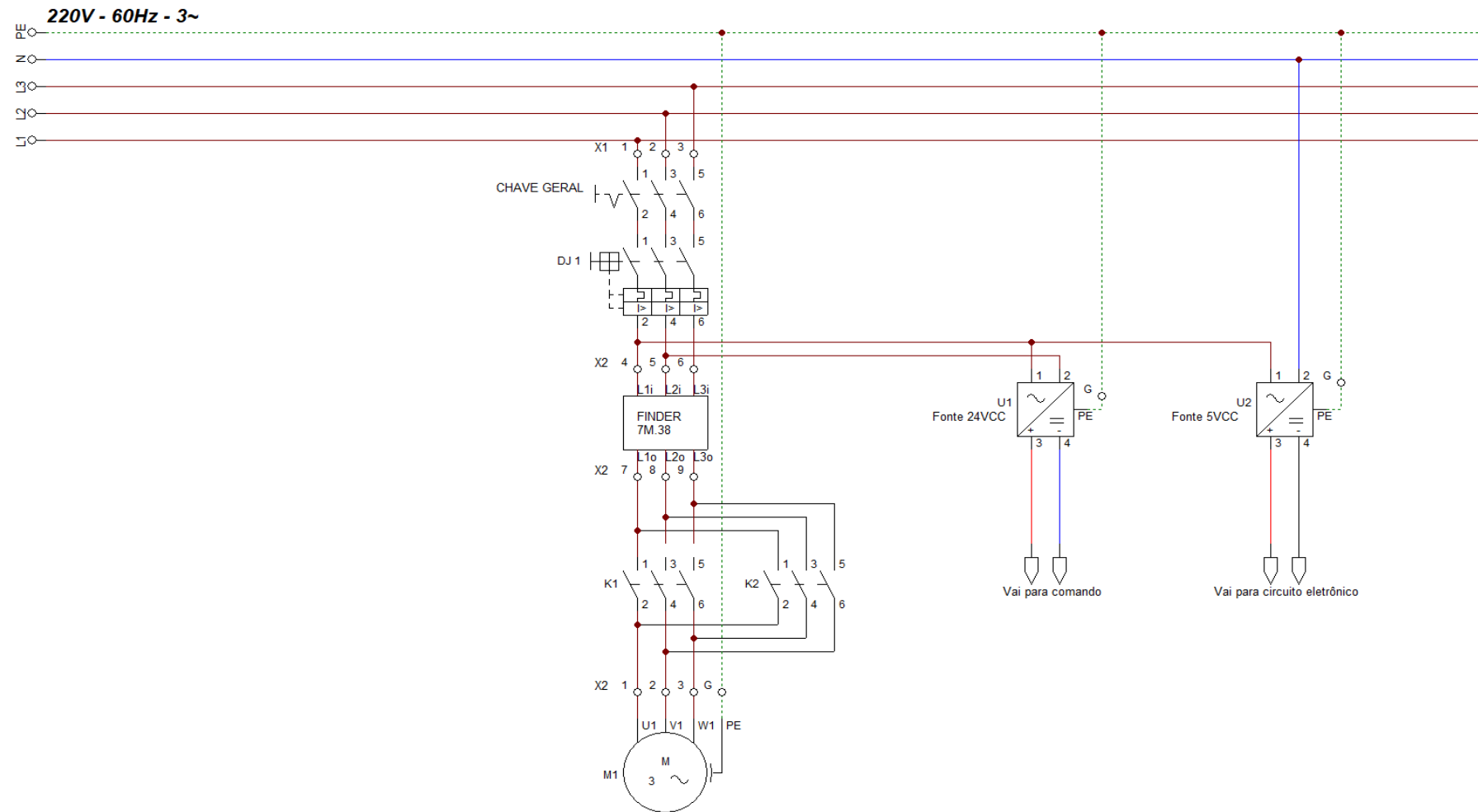
APÊNDICE D – PLACA PCB



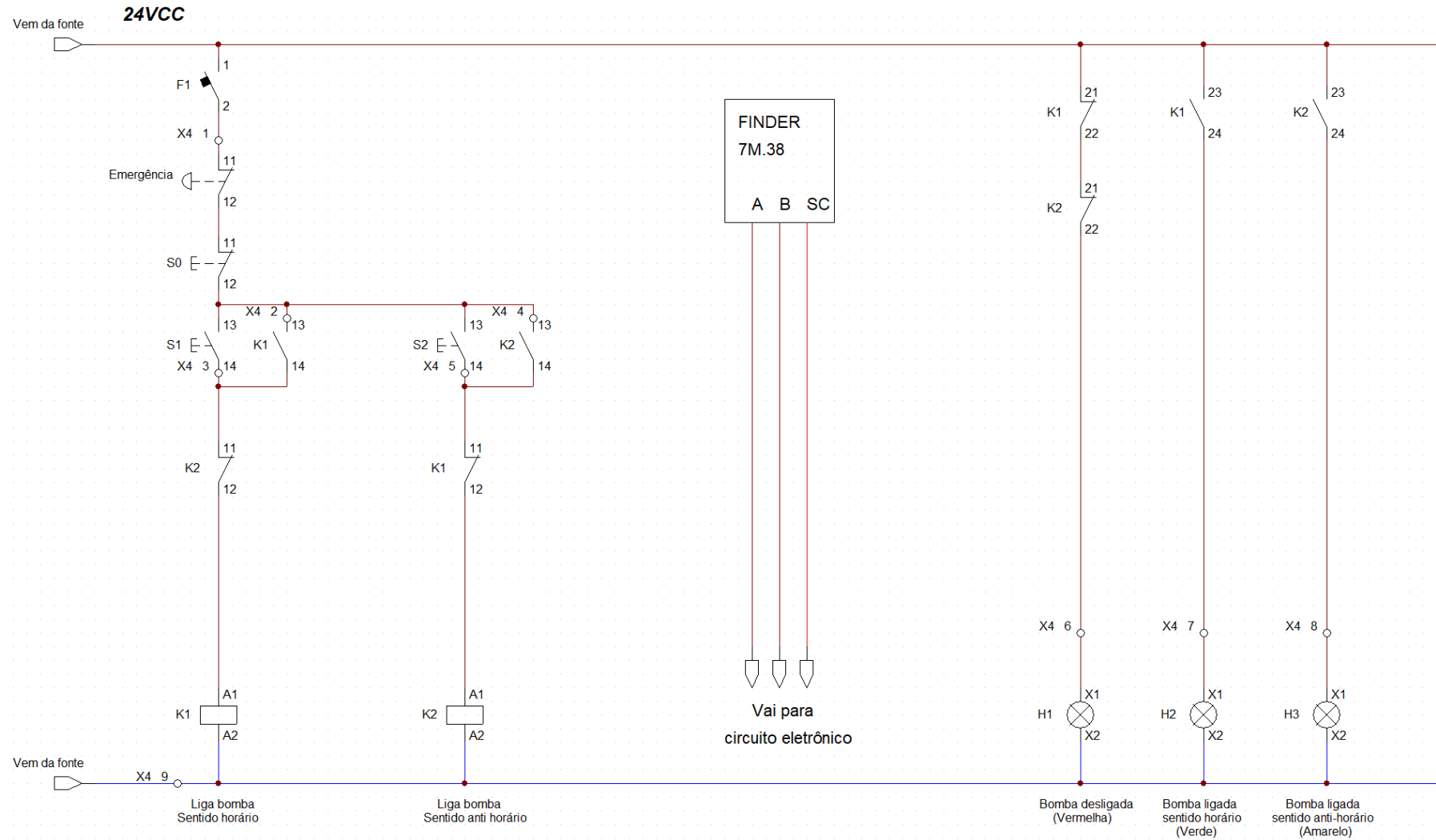
APÊNDICE E – PAINEL



APÊNDICE F – DIAGRAMA ELÉTRICO - POTÊNCIA



APÊNDICE G – DIAGRAMA ELÉTRICO - COMANDO



APÊNDICE H – CÓDIGO DA PROGRAMAÇÃO

```
#include <WiFi.h>
#include "RTCLib.h"
#include "time.h"
#include <ModbusMaster.h>
#include <SD.h>
#include <SPI.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

#define MQTT_MAX_PACKET_SIZE 1024

const char* ssid = "moto e6 play 9694";
const char* password = "batata05";
const char* mqtt_server = "10.170.194.225";
const int mqtt_port = 1883;

RTC_DS3231 rtc;
#define RXD2 16
#define TXD2 17
constexpr uint8_t MODBUS_7M_ADDRESS = 1;
ModbusMaster node;
#define SD_CS 5

WiFiClient espClient;
PubSubClient client(espClient);

const char* ntpServer = "pool.ntp.org";
const long gmtOffset_sec = -3 * 3600;
const int daylightOffset_sec = 0;
```

```

// FLAGS DE STATUS (agora verificadas com segurança)
bool rtc_ok = false;
bool sd_ok = false;

// ===== AJUSTAR HORA VIA NTP (só se WiFi conectado) =====
void ajustarViaNTP()
{
  if (WiFi.status() != WL_CONNECTED) return;
  Serial.println("Ajustando hora via NTP...");
  configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
  struct tm timeinfo;
  int tentativas = 0;
  while (!getLocalTime(&timeinfo) && tentativas < 10)
  {
    delay(1000); tentativas++;
  }
  if (getLocalTime(&timeinfo) && rtc_ok)
  {
    rtc.adjust(DateTime(timeinfo.tm_year + 1900, timeinfo.tm_mon + 1,
    timeinfo.tm_mday, timeinfo.tm_hour, timeinfo.tm_min, timeinfo.tm_sec));
    Serial.println("RTC ajustado via NTP com sucesso!");
  }
}

// ===== LER FLOAT IEEE754 =====
float readFloatIEEE(uint16_t reg)
{
  uint8_t result = node.readInputRegisters(reg, 2);
  if (result == node.ku8MBSuccess)
  {
    uint16_t w0 = node.getResponseBuffer(0);

```

```

uint16_t w1 = node.getResponseBuffer(1);
uint32_t rawHL = ((uint32_t)w0 << 16) | w1;
uint32_t rawLH = ((uint32_t)w1 << 16) | w0;
float fHL, fLH;
memcpy(&fHL, &rawHL, 4);
memcpy(&fLH, &rawLH, 4);
if (fHL > 0.1 && fHL < 80000) return fHL;
if (fLH > 0.1 && fLH < 80000) return fLH;
}
return 0;
}

// ===== GRAVAR NO SD (só tenta se SD estiver OK) =====
void salvarNoSD(const char* label, float value, const char* filename)
{
    if (!sd_ok) return; // SD falhou → pula silenciosamente
    DateTime now = rtc.now();
    if (now.year() < 2024) return; // data inválida → pula

    String dateStr = String(now.year()) + "-" +
        (now.month() < 10 ? "0" : "") + String(now.month()) + "-" +
        (now.day() < 10 ? "0" : "") + String(now.day());
    String timeStr = (now.hour() < 10 ? "0" : "") + String(now.hour()) + ":" +
        (now.minute() < 10 ? "0" : "") + String(now.minute()) + ":" +
        (now.second() < 10 ? "0" : "") + String(now.second());

    File file = SD.open(filename, FILE_APPEND);
    if (file)
    {
        if (file.size() == 0) file.println("Grandeza,Valor,Data,Hora");
        file.print(label); file.print(",");
    }
}

```

```

file.print(value, 2); file.print(",");
file.print(dateStr); file.print(",");
file.println(timeStr);
file.close();
}
}

// ===== RECONECTAR WIFI E MQTT =====

void reconectarWiFi()
{
if (WiFi.status() == WL_CONNECTED) return;
Serial.print("WiFi perdido! Reconnectando");
WiFi.disconnect();
WiFi.begin(ssid, password);
int t = 0;
while (WiFi.status() != WL_CONNECTED && t < 40) { delay(500); Serial.print(".");
t++;
}
Serial.println(WiFi.status() == WL_CONNECTED ? "\nWiFi reconectado!" : "\nWiFi
falhou");
}

void reconectarMQTT()
{
if (WiFi.status() != WL_CONNECTED || client.connected()) return;
int t = 0;
while (!client.connected() && t < 5)
{
Serial.print("Tentando MQTT...");
if (client.connect("ESP32_TCC"))
{
Serial.println(" MQTT CONECTADO!");
}
}
}

```

```

return;
}
delay(3000); t++;
}
if (!client.connected()) Serial.println(" MQTT FALHOU");
}

// ===== ENVIO DAS PARTES (inalterado) =====
void enviarParte1()
{
  StaticJsonDocument<1024> doc;
  DateTime now = rtc_ok ? rtc.now() : DateTime(2025, 11, 18, 12, 0, 0);
  char ts[20];
  sprintf(ts, "%04d-%02d-%02d %02d:%02d:%02d", now.year(), now.month(),
  now.day(),
  now.hour(), now.minute(), now.second());
  doc["ts"] = ts;

  doc["U1"] = readFloatIIEEE(32500 - 30001);
  doc["U2"] = readFloatIIEEE(32502 - 30001);
  doc["U3"] = readFloatIIEEE(32504 - 30001);
  doc["U12"] = readFloatIIEEE(32508 - 30001);
  doc["U23"] = readFloatIIEEE(32510 - 30001);
  doc["U31"] = readFloatIIEEE(32512 - 30001);
  doc["I1"] = readFloatIIEEE(32516 - 30001);
  doc["I2"] = readFloatIIEEE(32518 - 30001);
  doc["I3"] = readFloatIIEEE(32520 - 30001);
  doc["Iavg"] = readFloatIIEEE(32528 - 30001);

  char buffer[1024];
  size_t len = serializeJson(doc, buffer);

```

```

client.publish("Tcc/energia/parte1", buffer, len);
}

```

```

void enviarParte2()

```

```

{

```

```

    StaticJsonDocument<1024> doc;

```

```

    DateTime now = rtc_ok ? rtc.now() : DateTime(2025, 11, 18, 12, 0, 0);

```

```

    char ts[20];

```

```

    sprintf(ts, "%04d-%02d-%02d %02d:%02d:%02d", now.year(), now.month(),
    now.day(),

```

```

    now.hour(), now.minute(), now.second());

```

```

    doc["ts"] = ts;

```

```

    doc["P1"] = readFloatIeee(32530 - 30001); doc["P2"] = readFloatIeee(32532 -
    30001);

```

```

    doc["P3"] = readFloatIeee(32534 - 30001); doc["Pt"] = readFloatIeee(32536 -
    30001);

```

```

    doc["Q1"] = readFloatIeee(32538 - 30001); doc["Q2"] = readFloatIeee(32540 -
    30001);

```

```

    doc["Q3"] = readFloatIeee(32542 - 30001); doc["Qt"] = readFloatIeee(32544 -
    30001);

```

```

    doc["S1"] = readFloatIeee(32546 - 30001); doc["S2"] = readFloatIeee(32548 -
    30001);

```

```

    doc["S3"] = readFloatIeee(32550 - 30001); doc["St"] = readFloatIeee(32552 -
    30001);

```

```

    doc["PF1"] = readFloatIeee(32554 - 30001); doc["PF2"] = readFloatIeee(32556 -
    30001);

```

```

    doc["PF3"] = readFloatIeee(32558 - 30001); doc["PFt"] = readFloatIeee(32560 -
    30001);

```

```

    doc["ang_U1_I1"] = readFloatIeee(32570 - 30001); doc["ang_U2_I2"] =
    readFloatIeee(32572 - 30001);

```

```

    doc["ang_U3_I3"] = readFloatIeee(32574 - 30001); doc["ang_total"] =
    readFloatIeee(32576 - 30001);

```

```
doc["ang_U1_U2"] = readFloatIEEE(32578 - 30001); doc["ang_U2_U3"] =
readFloatIEEE(32580 - 30001);
```

```
doc["ang_U3_U1"] = readFloatIEEE(32582 - 30001);
```

```
doc["freq"] = readFloatIEEE(32584 - 30001);
```

```
doc["THD_I1"] = readFloatIEEE(32588 - 30001); doc["THD_I2"] =
readFloatIEEE(32590 - 30001);
```

```
doc["THD_I3"] = readFloatIEEE(32592 - 30001);
```

```
doc["THD_U1"] = readFloatIEEE(32594 - 30001); doc["THD_U2"] =
readFloatIEEE(32596 - 30001);
```

```
doc["THD_U3"] = readFloatIEEE(32598 - 30001);
```

```
doc["runtime"] = readFloatIEEE(32480 - 30001);
```

```
doc["Kwh1"] = readFloatIEEE(32638 - 30001); doc["Kwh2"] = readFloatIEEE(32640 -
30001);
```

```
doc["Kvarh3"] = readFloatIEEE(32642 - 30001); doc["Kvarh4"] =
readFloatIEEE(32644 - 30001);
```

```
char buffer[1024];
```

```
size_t len = serializeJson(doc, buffer);
```

```
client.publish("Tcc/energia/parte2", buffer, len);
```

```
}
```

```
// ===== SETUP - NUNCA MAIS TRAVA =====
```

```
void setup()
```

```
{
```

```
Serial.begin(115200);
```

```
delay(2000);
```

```
Serial.println("\n\n=== TCC RODRIGO - VERSÃO INDESTRUTÍVEL ===\n");
```

```
Serial2.begin(38400, SERIAL_8N1, RXD2, TXD2);
```

```
node.begin(MODBUS_7M_ADDRESS, Serial2);
```

```
Serial.println("Modbus RTU iniciado (OK)");
```

```
// TESTE DO RTC (não trava mais)
```

```
if (!rtc.begin())
```

```
{
```

```
Serial.println(">>> MÓDULO RTC (DS3231) FALHOU OU DESCONECTADO <<<");
```

```
rtc_ok = false;
```

```
}
```

```
else
```

```
{
```

```
Serial.println("Módulo RTC detectado (OK)");
```

```
rtc_ok = true;
```

```
}
```

```
// TESTE DO SD (não trava mais)
```

```
if (!SD.begin(SD_CS))
```

```
{
```

```
Serial.println(">>> MÓDULO SD OU CARTÃO FALHOU (continuando sem salvar)  
<<<");
```

```
sd_ok = false;
```

```
}
```

```
else
```

```
{
```

```
Serial.println("Cartão SD detectado (OK)");
```

```
sd_ok = true;
```

```
}
```

```
WiFi.begin(ssid, password);
```

```
int t = 0;
```

```
Serial.print("Conectando WiFi");
```

```

while (WiFi.status() != WL_CONNECTED && t < 40) { delay(500); Serial.print(".");
t++;
}

Serial.println(WiFi.status() == WL_CONNECTED ?

    "\nWiFi conectado! IP: " + WiFi.localIP().toString() :

    "\nWiFi não conectou");

if (WiFi.status() == WL_CONNECTED)
{
    ajustarViaNTP();
    client.setServer(mqtt_server, mqtt_port);
    client.setBufferSize(1024);
    reconectarMQTT();
}

Serial.println("\n=== SISTEMA INICIADO COM SUCESSO - RODANDO 100%
===\n");
}

// ===== LOOP COM DIAGNÓSTICO E MEDIÇÕES NO SERIAL =====
void loop()
{
    if (WiFi.status() != WL_CONNECTED) reconectarWiFi();
    if (WiFi.status() == WL_CONNECTED && !client.connected()) reconectarMQTT();
    client.loop();

    static unsigned long last = 0;
    if (millis() - last < 10000) return;
    last = millis();

    // DIAGNÓSTICO ANTES DE CADA LEITURA
    Serial.println("\n>>> NOVA LEITURA A CADA 10 SEGUNDOS <<<");
    Serial.printf("DIAGNÓSTICO → RTC: %s | SD: %s\n",
        rtc_ok ? "OK" : "FALHOU",

```

```
sd_ok ? "OK" : "FALHOU");
```

```
Serial.println("=====
```

```
=====");
```

```
float U1 = readFloatIIEEE(32500 - 30001); Serial.printf("U1 (F-N):   %.2f V\n", U1);
salvarNoSD("U1", U1, "/tensao.csv");
```

```
float U2 = readFloatIIEEE(32502 - 30001); Serial.printf("U2 (F-N):   %.2f V\n", U2);
salvarNoSD("U2", U2, "/tensao.csv");
```

```
float U3 = readFloatIIEEE(32504 - 30001); Serial.printf("U3 (F-N):   %.2f V\n", U3);
salvarNoSD("U3", U3, "/tensao.csv");
```

```
float U12 = readFloatIIEEE(32508 - 30001); Serial.printf("U12 (L-L):   %.2f V\n",
U12); salvarNoSD("U12", U12, "/tensao_linha.csv");
```

```
float U23 = readFloatIIEEE(32510 - 30001); Serial.printf("U23 (L-L):   %.2f V\n",
U23); salvarNoSD("U23", U23, "/tensao_linha.csv");
```

```
float U31 = readFloatIIEEE(32512 - 30001); Serial.printf("U31 (L-L):   %.2f V\n",
U31); salvarNoSD("U31", U31, "/tensao_linha.csv");
```

```
float I1 = readFloatIIEEE(32516 - 30001); Serial.printf("I1:           %.3f A\n", I1);
salvarNoSD("I1", I1, "/corrente.csv");
```

```
float I2 = readFloatIIEEE(32518 - 30001); Serial.printf("I2:           %.3f A\n", I2);
salvarNoSD("I2", I2, "/corrente.csv");
```

```
float I3 = readFloatIIEEE(32520 - 30001); Serial.printf("I3:           %.3f A\n", I3);
salvarNoSD("I3", I3, "/corrente.csv");
```

```
float lavg = readFloatIIEEE(32528 - 30001); Serial.printf("I média:      %.3f A\n",
lavg); salvarNoSD("lavg", lavg, "/corrente.csv");
```

```
float P1 = readFloatIIEEE(32530 - 30001); Serial.printf("P1 Ativa:      %.2f kW\n",
P1/1000); salvarNoSD("P1", P1, "/potencia_ativa.csv");
```

```
float P2 = readFloatIIEEE(32532 - 30001); Serial.printf("P2 Ativa:      %.2f kW\n",
P2/1000); salvarNoSD("P2", P2, "/potencia_ativa.csv");
```

```
float P3 = readFloatIIEEE(32534 - 30001); Serial.printf("P3 Ativa:      %.2f kW\n",
P3/1000); salvarNoSD("P3", P3, "/potencia_ativa.csv");
```

```
float Pt = readFloatIIEEE(32536 - 30001); Serial.printf("P Total:      %.2f kW\n",
Pt/1000); salvarNoSD("Pt", Pt, "/potencia_ativa.csv");
```

```
float Q1 = readFloatIIEEE(32538 - 30001); Serial.printf("Q1 Reativa:   %.2f kvar\n",
Q1/1000); salvarNoSD("Q1", Q1, "/potencia_reativa.csv");
```

```
float Q2 = readFloatIEEE(32540 - 30001); Serial.printf("Q2 Reativa:  %.2f kvar\n",
Q2/1000); salvarNoSD("Q2", Q2, "/potencia_reativa.csv");
```

```
float Q3 = readFloatIEEE(32542 - 30001); Serial.printf("Q3 Reativa:  %.2f kvar\n",
Q3/1000); salvarNoSD("Q3", Q3, "/potencia_reativa.csv");
```

```
float Qt = readFloatIEEE(32544 - 30001); Serial.printf("Q Total:    %.2f kvar\n",
Qt/1000); salvarNoSD("Qt", Qt, "/potencia_reativa.csv");
```

```
float S1 = readFloatIEEE(32546 - 30001); Serial.printf("S1 Aparente:  %.2f kVA\n",
S1/1000); salvarNoSD("S1", S1, "/potencia_aparente.csv");
```

```
float S2 = readFloatIEEE(32548 - 30001); Serial.printf("S2 Aparente:  %.2f kVA\n",
S2/1000); salvarNoSD("S2", S2, "/potencia_aparente.csv");
```

```
float S3 = readFloatIEEE(32550 - 30001); Serial.printf("S3 Aparente:  %.2f kVA\n",
S3/1000); salvarNoSD("S3", S3, "/potencia_aparente.csv");
```

```
float St = readFloatIEEE(32552 - 30001); Serial.printf("S Total:    %.2f kVA\n",
St/1000); salvarNoSD("St", St, "/potencia_aparente.csv");
```

```
float PF1 = readFloatIEEE(32554 - 30001); Serial.printf("FP Fase 1:   %.3f\n",
PF1); salvarNoSD("PF1", PF1, "/fator_potencia.csv");
```

```
float PF2 = readFloatIEEE(32556 - 30001); Serial.printf("FP Fase 2:   %.3f\n",
PF2); salvarNoSD("PF2", PF2, "/fator_potencia.csv");
```

```
float PF3 = readFloatIEEE(32558 - 30001); Serial.printf("FP Fase 3:   %.3f\n",
PF3); salvarNoSD("PF3", PF3, "/fator_potencia.csv");
```

```
float PFt = readFloatIEEE(32560 - 30001); Serial.printf("FP Total:    %.3f\n", PFt);
salvarNoSD("PFT", PFT, "/fator_potencia.csv");
```

```
float ang1 = readFloatIEEE(32570 - 30001); Serial.printf("Ângulo U1-I1: %.1f °\n",
ang1); salvarNoSD("ang_U1_I1", ang1, "/angulos.csv");
```

```
float ang2 = readFloatIEEE(32572 - 30001); Serial.printf("Ângulo U2-I2: %.1f °\n",
ang2); salvarNoSD("ang_U2_I2", ang2, "/angulos.csv");
```

```
float ang3 = readFloatIEEE(32574 - 30001); Serial.printf("Ângulo U3-I3: %.1f °\n",
ang3); salvarNoSD("ang_U3_I3", ang3, "/angulos.csv");
```

```
float angT = readFloatIEEE(32576 - 30001); Serial.printf("Ângulo Total: %.1f °\n",
angT); salvarNoSD("ang_total", angT, "/angulos.csv");
```

```
float ang12 = readFloatIEEE(32578 - 30001); Serial.printf("Ângulo U1-U2: %.1f °\n",
ang12); salvarNoSD("ang_U1_U2", ang12, "/angulos.csv");
```

```
float ang23 = readFloatIEEE(32580 - 30001); Serial.printf("Ângulo U2-U3: %.1f °\n",
ang23); salvarNoSD("ang_U2_U3", ang23, "/angulos.csv");
```

```
float ang31 = readFloatIEEE(32582 - 30001); Serial.printf("Ângulo U3-U1: %.1f °\n",
ang31); salvarNoSD("ang_U3_U1", ang31, "/angulos.csv");
```

```
float freq = readFloatIEEE(32584 - 30001); Serial.printf("Frequência: %.2f Hz\n",
freq); salvarNoSD("Frequência", freq, "/frequencia.csv");
```

```
float thd_i1 = readFloatIEEE(32588 - 30001); Serial.printf("THD I1: %.2f%%\n",
thd_i1); salvarNoSD("THD_I1", thd_i1, "/thd_correntes.csv");
```

```
float thd_i2 = readFloatIEEE(32590 - 30001); Serial.printf("THD I2: %.2f%%\n",
thd_i2); salvarNoSD("THD_I2", thd_i2, "/thd_correntes.csv");
```

```
float thd_i3 = readFloatIEEE(32592 - 30001); Serial.printf("THD I3: %.2f%%\n",
thd_i3); salvarNoSD("THD_I3", thd_i3, "/thd_correntes.csv");
```

```
float thd_u1 = readFloatIEEE(32594 - 30001); Serial.printf("THD U1:
%.2f%%\n", thd_u1); salvarNoSD("THD_U1", thd_u1, "/thd_tensoes.csv");
```

```
float thd_u2 = readFloatIEEE(32596 - 30001); Serial.printf("THD U2:
%.2f%%\n", thd_u2); salvarNoSD("THD_U2", thd_u2, "/thd_tensoes.csv");
```

```
float thd_u3 = readFloatIEEE(32598 - 30001); Serial.printf("THD U3:
%.2f%%\n", thd_u3); salvarNoSD("THD_U3", thd_u3, "/thd_tensoes.csv");
```

```
float runtime = readFloatIEEE(32480 - 30001); Serial.printf("Tempo ligado: %.2f
h\n", runtime); salvarNoSD("Runtime", runtime, "/runtime.csv");
```

```
float kwh1 = readFloatIEEE(32638 - 30001); Serial.printf("kWh Fase 1: %.2f
kWh\n", kwh1); salvarNoSD("Kwh1", kwh1, "/energia.csv");
```

```
float kwh2 = readFloatIEEE(32640 - 30001); Serial.printf("kWh Fase 2: %.2f
kWh\n", kwh2); salvarNoSD("Kwh2", kwh2, "/energia.csv");
```

```
float kvarh3 = readFloatIEEE(32642 - 30001); Serial.printf("kvarh 3: %.2f
kvarh\n", kvarh3); salvarNoSD("Kvarh3", kvarh3, "/energia.csv");
```

```
float kvarh4 = readFloatIEEE(32644 - 30001); Serial.printf("kvarh 4: %.2f
kvarh\n", kvarh4); salvarNoSD("Kvarh4", kvarh4, "/energia.csv");
```

```
Serial.println("=====
```

```
=====");
if (client.connected())
```

```
{
```

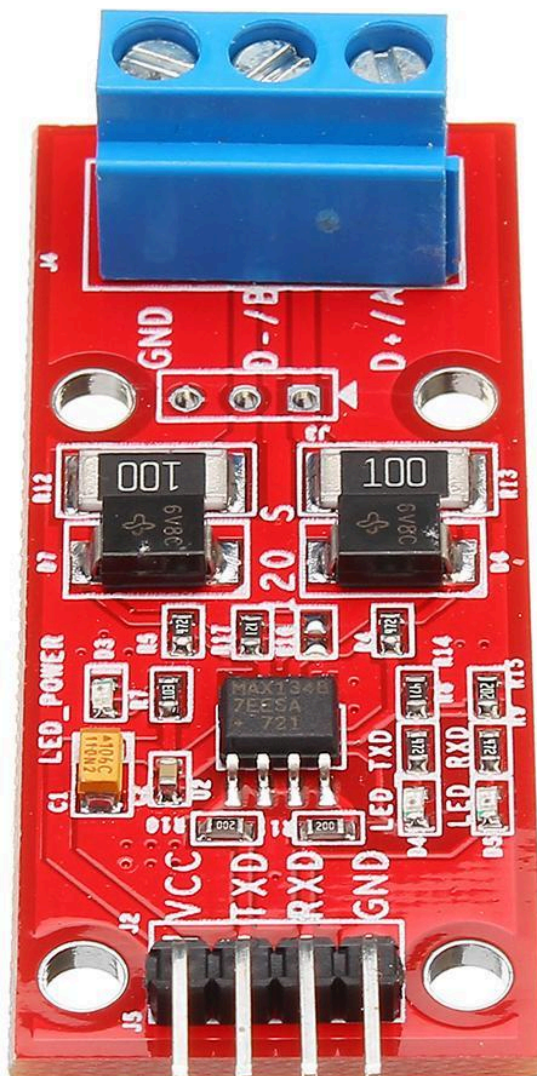
```
enviarParte1();
```

```
delay(100);
```

```
enviarParte2();  
Serial.println("2 PARTES JSON ENVIADAS COM SUCESSO!");  
}  
else  
{  
Serial.println("MQTT DESCONECTADO - Dados apenas no Serial/SD");  
}  
}
```

ANEXO A – FINDER 7M38.8.400.0212

Fonte: Conrad Electronic (2025).

ANEXO C – MAX3485

Fonte: EletronicPro (2025)

ANEXO D – MÓDULO RTC (REAL TIME CLOCK)



Fonte: Autocore (2025)

ANEXO E – MÓDULO MICROSD



Fonte: Saravati (2025)

